Contents

Go Quick Tutorial

A 2-hour complete Go course.

A fast, dense tutorial for experienced developers who need to learn Go. Not a reference book — you have the docs for that. Not a beginner guide — you already know what a type, a pointer, or a closure is.

Each chapter covers exactly one concept, builds on the previous ones, and moves on. The learning curve is gentle but constant: no padding, no detours, no noise. The structure is designed for memorization — clear sections, bold syntax, focused examples — so you can scan, retain, and move forward.

Audience: Proficient developers who understand programming language fundamentals and need Go's specific syntax, choices, and idioms. Fast.

01 — Program Structure

Go organizes code into packages, where every source file declares its membership and the compiler enforces strict boundaries between them.

Every `.go` file begins with a package declaration:

```
package main
```

Files in the same directory must share the same package name and are compiled together as a single unit. A name defined in one file is visible to all other files in the same package without any import.

By convention, the package name matches the directory name. Files in a `geometry` directory declare `package geometry`. `go vet` warns when they differ.

The entry point of an executable program is the function `main` in `package main`:

```
package main

func main() {
}
```

Visibility across package boundaries is controlled entirely by capitalization. An identifier starting with an uppercase letter is exported — accessible from other packages. One starting with a lowercase letter is unexported — accessible within the package across all its files, but invisible outside.

```
package geometry

var Pi = 3.14159      // exported
var precision = 10    // unexported

func Area(r float64) float64 { ... }   // exported
func validate(r float64) bool { ... }  // unexported
```

There are no `public`, `private`, or `protected` keywords. Capitalization is the complete visibility system. Unexported does not mean private to a single file — every file in the package can use it.

Semicolons

Go does not require a semicolon at the end of each statement. The lexer automatically inserts semicolons at newlines in specific situations — primarily after a statement when the line ends with an expression, an identifier, or certain keywords (`break`, `continue`, `fallthrough`, `return`) or tokens (`++`, `--`, `)`, `]`, `}`).

In practice, this means you write code without semicolons and it works. The rules matter when you need to break a statement across lines or put two statements on one line:

```go
for i := 0; i < 10; i++ {
    fmt.Println(i)
}  // semicolons required inside for

x := 1; y := 2  // two statements on one line need a semicolon
```

The automatic insertion means an opening `{` must be on the same line as the preceding keyword (`if`, `for`, `func`, `switch`, etc.), otherwise the lexer inserts a semicolon before it, creating an empty statement.

```go
if x > 0 {  // correct
    fmt.Println("positive")
}

if x > 0
{  // compile error: { treated as start of a new statement
    fmt.Println("positive")
}
```

A newline does NOT trigger semicolon insertion when the line ends with an operator, an opening token (`(`, `{`, `[`), a comma, or the keywords `case` or `default`. This is what allows breaking long expressions across lines:

```go
result = a +
    b +
    c

if (x > 0 &&
    y < 10) {
    fmt.Println("range")
}

config := Config{
    Host: "localhost",
    Port: 8080,
}
```

For method chains, the dot must be at the end of the line, not the beginning of the next:

```go
person.address("main").  // OK - line ends with .
    number

person.address("main")   // ERROR - line ends with ), semicolon inserted
```

```
    .number
```

02 — Imports

Go's import system brings external packages into scope, with strict rules the compiler enforces.

Import Syntax

Single Import

```
import "fmt"
```

Used in a complete program:

```
package main

import "fmt"

func main() {
    fmt.Println("hello")
}
```

Grouped Imports

Multiple imports use a grouped block — the **preferred form** in all real code:

```
import (
    "fmt"
    "os"
    "net/http"
)
```

Import Path Categories

Category	Example	Description
Standard library	`"fmt"`, `"os"`, `"net/http"`	Ships with Go, requires no setup
External packages	`"github.com/example/json"`	Fetched with `go get` — mechanics covered in the next chapter

---

Package Naming

The local name of an imported package is its **last path element** by default:

- `"net/http"` → used as `http`
- `"encoding/json"` → used as `json`
- `"path/filepath"` → used as `filepath`

When two imports share a last element, or when a shorter alias is clearer, declare one explicitly:

```
import (
    "fmt"
    myjson "github.com/example/json"  // referenced as myjson in this file
)
```

Special Import Forms

Blank Import

Runs a package's initialization code without making any of its names available. Standard for database drivers and image format decoders, which register themselves as a side effect:

```
import _ "github.com/lib/pq"  // registers PostgreSQL driver, nothing else used
```

Dot Import

Pulls all exported names directly into the current file's namespace, removing the need for a qualifier. With `import . "fmt"`, calls like `fmt.Println("hello")` become `Println("hello")`. It exists in the language but *obscures where names originate* and is rarely appropriate:

```
import . "fmt"  // Println is now available without the fmt. prefix
```

Unused Imports

The compiler treats an unused import as an **error, unconditionally**. If a package is imported and nothing from it is referenced, the program does not compile. There is no flag to suppress this.

Command-Line Arguments

Command-line arguments are available through `os.Args`, which returns an array of strings:

- `os.Args[0]` — the program name

- `os.Args[1]`, `os.Args[2]`, etc. — the user-supplied arguments

```go
package main

import (
    "fmt"
    "os"
)

func main() {
    fmt.Println(os.Args[0])  // program name, e.g. "./myapp"
    fmt.Println(os.Args[1])  // first argument, e.g. "hello"
    fmt.Println(os.Args[2])  // second argument, e.g. "world"
}
```

03 — Packages and Modules

Go's module system defines the unit of versioning and distribution, layered above individual packages.

Packages

A **package** is a directory of `.go` files sharing the same `package` declaration.

Naming conventions:

- Short, lowercase, no underscores
- Conventionally matches the directory name
- `go vet` warns when package name and directory name differ

Key rules:

- Two packages cannot import each other — *circular imports are a compile error*
- Any package with exported names is importable by its path — there is no separate "export" concept

Internal Packages

A directory named `internal` restricts which other packages can import its contents. Conventionally, `internal` contains subdirectories, each a separate package — `internal/auth/` is imported as `"example/myapp/internal/auth"`. It is also possible to place `.go` files directly in `internal/` (making it a package itself, importable as `"example/myapp/internal"`), but this is uncommon. Only code within the same module can import these packages. A different module trying to import `"example/myapp/internal/auth"` is rejected by the compiler.

A nested `internal` directory applies the same rule at a smaller scope. `storage/internal/cache` is importable only by packages inside `storage/` and its subdirectories — not even by other packages in the same module.

Modules

A **module** is a tree of packages rooted at a directory containing a `go.mod` file. The module path declared in `go.mod` becomes the import prefix for every package in the module:

```
module github.com/example/myapp

go 1.22

require (
```

```
    github.com/some/library v1.4.2
)
```

A package at `myapp/storage/postgres` is imported as `"github.com/example/myapp/storage/postgres"`.

Program vs Library

There is **no formal distinction** between a program module and a library module:

- If any package declares `package main` with a `main()` function, `go build` produces an *executable*
- Packages that are not `package main` are *library packages* — importable by other modules
- A single module can be **both**: contain importable library packages and a `main` package that builds an executable

---

Dependencies

Automatic Dependency Download (Preferred)

The typical workflow is: **add the import to your code**, then run `go mod tidy`. Go automatically:

1. Resolves the module path from the import statement
2. Downloads the module if it is not cached locally
3. Adds the dependency to `go.mod` with the minimum required version
4. Updates `go.sum` with the cryptographic hash

Example — adding a GitHub dependency:

Add an import to your code:

```
import "github.com/gorilla/mux"
```

Run `go mod tidy`. Go resolves the module path, downloads the latest compatible version, and updates `go.mod`:

```
require github.com/gorilla/mux v1.8.1
```

The module is now cached locally and available for building. The same automatic resolution happens when you run `go build` or `go run` — they will download missing dependencies before compiling.

You cannot specify a version with automatic download. `go mod tidy` always picks the minimum version required by your code. If you need a specific version, use `go get` instead.

Other Dependency Methods

`go get` — explicit version control:

```
go get github.com/gorilla/mux@v1.8.1
```

Use `go get` when you need to pin a specific version, upgrade beyond the current minimum, or add a dependency before writing the import.

`go mod download` — download without modifying `go.mod`:

```
go mod download
```

Downloads all dependencies listed in `go.mod` to the local cache without adding or removing entries. Useful for pre-populating the cache in CI pipelines or verifying that all dependencies are reachable.

`go mod vendor` — local vendor directory:

```
go mod vendor
```

Copies all dependencies into a `vendor/` directory within your project. Subsequent builds use the vendored copies instead of downloading. Useful for air-gapped environments or when you want to lock the exact source code your project builds against.

`go mod why` — understand why a dependency is needed:

```
go mod why github.com/uber/atomic
```

Prints the shortest import chain from your main module to the specified dependency, showing which of your packages transitively requires it.

Removing Dependencies

`go mod tidy` removes unused dependencies automatically. To remove a specific one:

```
go get github.com/some/library@none
```

This removes the entry from `go.mod` entirely.

Semver Versioning

Versions map to git tags in semver format: `v<major>.<minor>.<patch>`. In `v1.4.2`:

- **Major** (`1`) — breaking changes. Incrementing to `v2.0.0` signals incompatibility with `v1.x.x`
- **Minor** (`4`) — backward-compatible additions. `v1.5.0` adds features without breaking `v1.4.x` code
- **Patch** (`2`) — bug fixes. `v1.4.3` fixes bugs without changing behavior

How Go resolves versions:

- `go.mod` records a *minimum* version, not a pinned version. If `go.mod` says `v1.4.2`, and another dependency requires `v1.4.5`, Go uses `v1.4.5`
- You cannot request "latest patch within a minor version" directly. Go's minimum version selection handles this automatically — the highest required version within the same major is always used

- To get the latest version of a dependency regardless of current requirements: `go get github.com/some/library@latest`

Direct vs Indirect Dependencies

Type	Description	Example
Direct	A package your code imports explicitly	`github.com/some/library v1.4.2`
Indirect	A dependency of a dependency	`github.com/other/utils v0.3.1 // indirect`

Both appear in `go.mod`, with indirect ones marked `// indirect`.

Version Conflict Resolution

When two dependencies require different versions of the same transitive package, Go uses **minimum version selection** — it picks the *highest version required by any dependency*. This ensures a single version of each module is used throughout the build.

Dependency Files

- `go.mod` — declares the module path, minimum Go version, and dependencies
- `go.sum` — records cryptographic hashes of every downloaded module
- *Commit both files* to version control

Standard Library

Standard library packages are part of the Go installation:

- Require no `go get`
- Appear in no `go.mod`

Major Version Suffixes

When a module releases a breaking v2 version, the module path gains a **version suffix**: `github.com/example/lib/v2`.

Rules:

- Applies to all versions >= **v2**: `/v3`, `/v4`, etc.
- The suffix appears in the *module path* and in *all import paths*
- v1 modules have no suffix
- Allows multiple major versions of the same module to coexist as distinct imports

Build Tags

Build tags control which `.go` files are included in a package compilation. A tag directive at the top of a file tells the compiler to include or exclude the file based on conditions like OS, architecture, or custom labels.

```
//go:build linux

package main
```

This file is compiled only on Linux. The blank line after the directive is required.

Common tag conditions:

Tag	Meaning
`linux`, `darwin`, `windows`	Operating system
`amd64`, `arm64`	CPU architecture
`go1.21`	Minimum Go version
Custom labels	Set with `go build -tags labelname`

Combining conditions:

```
//go:build linux && amd64
//go:build linux || darwin
//go:build !windows
```

Use `&&` (AND), `||` (OR), and `!` (NOT) to compose conditions.

File naming convention: Files can also be filtered by suffix — `file_linux.go` is only compiled on Linux, `file_test.go` is only included during `go test`. Build tags offer more flexibility than naming conventions alone.

Build Commands

Command	What it does
`go run .`	Compiles and executes a `main` package in one step — writes the binary to a *temporary location* and runs it directly, producing **no persistent artifact**
`go build`	Compiles the current package and its dependencies into a binary (for `main`) or validates (for libraries)

Command	What it does
`go install`	Builds and installs the resulting binary to `GOBIN` (default `~/go/bin`)
`go vet`	Runs static analysis to detect likely bugs and suspicious constructs

04 — Standard Library and Help System

Go ships with a large standard library and built-in tooling for navigating documentation — both online and offline.

Standard Library Overview

The standard library is extensive. It covers the areas below without requiring any external dependencies:

Area	Packages
Formatting and I/O	`fmt`, `io`, `os`, `bufio`
Networking	`net`, `net/http`
Encoding	`encoding/json`, `encoding/xml`, `encoding/csv`
String manipulation	`strings`, `strconv`, `unicode`, `regexp`
Concurrency	`sync`, `sync/atomic`
Time and dates	`time`
File paths	`path`, `path/filepath`
Testing	`testing`
Cryptography	`crypto/*`
Compression	`compress/*`, `archive/*`

The standard library source code is installed locally with Go and is readable — it is written in idiomatic Go and serves as a practical reference for patterns and conventions.

`go doc` — Offline Documentation

`go doc` prints package documentation in the terminal. It works offline and is fast — it is the first tool to reach for when looking up an API.

Package documentation

```
go doc net/http
```

Prints the package comment, exported types, and functions for `net/http`.

Specific symbol

```
go doc net/http.ListenAndServe
```

Prints the signature and documentation for a specific function, type, or constant.

Method on a type

```
go doc net/http.ResponseWriter.Write
```

Prints documentation for a method on a specific type.

`pkg.go.dev` — Web Documentation

`pkg.go.dev` is the web interface for all Go documentation. It covers:

- Every standard library package
- Every published third-party module

The URL pattern is predictable: `pkg.go.dev/std` for the standard library index, `pkg.go.dev/net/http` for a specific package, `pkg.go.dev/github.com/gin-gonic/gin` for a third-party module.

Doc-Aware Comments

Go's documentation tooling reads specially formatted comments. The first sentence of a comment is treated as a summary. Comments must start with the identifier name.

```go
// ListenAndServe starts an HTTP server with a given address and handler.
// It returns an error if the address is invalid.
func ListenAndServe(addr string, handler Handler) error {
    // ...
}
```

The `go doc` output shows "ListenAndServe starts an HTTP server with a given address and handler." as the summary line. Blank lines in the comment separate paragraphs in the rendered output.

This convention applies to packages (via `doc.go` files), types, functions, methods, and variables.

Navigation Strategy

When you need to find something in the standard library:

1. **Know the package name** — use `go doc packagename`

2. **Know the function name** — use `go doc packagename.FunctionName`
3. **Exploring** — browse `pkg.go.dev/std` or read the source locally

The standard library is organized by domain, not by abstraction level. Related functionality lives in the same package: `strings` contains all string operations, `os` contains all operating-system-level I/O, `net/http` contains the complete HTTP client and server implementation.

05 — Variables and Types

Go declares variables explicitly and infers types from assigned values. Every variable has a type that cannot change after declaration, and every variable is initialized to a zero value — there is no uninitialized state.

Variable Declaration

Three forms exist for declaring variables:

```
var name string = "Alice"  // explicit declaration
var name = "Alice"         // type inferred from value
name := "Alice"            // short declaration, inside functions only
```

The `var` form works at package level and inside functions. The `:=` short declaration works only inside functions — it is a compile error at package level.

Identifier Rules

An identifier is a sequence of letters, digits, and underscores that does not start with a digit. Go accepts Unicode letters, but the convention is ASCII-only: use `a-z`, `A-Z`, `0-9`, and `_`. Non-ASCII identifiers compile but will mark your code as unusual and can cause tooling issues.

Multiple variables in one statement:

```
var (
    host string = "localhost"
    port int    = 8080
)

host, port := "localhost", 8080
```

Zero Values

Every type has a zero value. Variables are never uninitialized:

```
var count int       // 0
var ratio float64   // 0.0
var flag bool       // false
var name string     // ""
var ptr *int        // nil
var slice []int     // nil
```

This eliminates a whole class of bugs from reading uninitialized memory. A declared variable is always usable immediately.

Type System

Go is statically typed. Once a variable's type is set, it cannot change:

```
var count int = 5
count = 10        // OK, same type
count = "ten"     // compile error
```

Type conversions are always explicit — there is no implicit coercion between types:

```
var count int = 5
var ratio float64 = float64(count)  // explicit conversion

var count int = 5
ratio := float64(count)             // inferred as float64
```

Numeric Literals

Integers can be written in multiple bases:

```
x := 42        // decimal (base 10)
x := 0x2A      // hexadecimal (base 16)
x := 0o52      // octal (base 8)
x := 0b101010  // binary (base 2)
```

Underscores improve readability of long literals:

```
x := 1_000_000
mask := 0b1111_0000
```

Floating-point literals support exponential notation:

```
x := 1.23e4   // 12300.0
x := 1.23e-4  // 0.000123
x := 123e2    // 12300.0
```

Numeric Types

Go provides a full set of integer, floating-point, and complex types. The size of `int`, `uint`, and `uintptr` depends on the architecture (32-bit or 64-bit).

Type	Size	Range / Description
`int`	32 or 64 bits	Default integer type
`int8`	8 bits	-128 to 127
`int16`	16 bits	-32768 to 32767

Type	Size	Range / Description
`int32`	32 bits	-2147483648 to 2147483647
`int64`	64 bits	-9223372036854775808 to 9223372036854775807
`uint`	32 or 64 bits	Unsigned, architecture-dependent
`uint8`	8 bits	0 to 255
`uint16`	16 bits	0 to 65535
`uint32`	32 bits	0 to 4294967295
`uint64`	64 bits	0 to 18446744073709551615
`uintptr`	32 or 64 bits	Large enough to hold a pointer bit pattern
`float32`	32 bits	IEEE-754 single precision
`float64`	64 bits	Default floating-point type
`complex64`	64 bits	float32 real + float32 imaginary
`complex128`	128 bits	float64 real + float64 imaginary

`int` is the default integer type. `float64` is the default floating-point type. Use the sized types (`int32`, `uint64`, etc.) when interfacing with fixed-width data formats, network protocols, or when you need guaranteed size across architectures.

`uintptr` is used with the `unsafe` package to perform pointer arithmetic. It holds the bit pattern of a pointer as an unsigned integer, but does not keep the pointed-to value alive — the garbage collector does not track `uintptr` values.

Aliases

Two aliases appear throughout Go code:

```
var b byte  // alias for uint8
var r rune  // alias for int32, represents a Unicode code point
```

`byte` is used when working with raw byte data. `rune` is used when iterating over Unicode text.

Bool

```
var active bool = true
var active bool = false
```

`bool` values are `true` and `false`. A `bool` cannot be converted from or to any other type — there is no implicit conversion from `0` to `false` or from a non-zero integer to `true`.

String

```
var greeting string = "Hello"
var greeting string = `Multi-line
string with "quotes"`
```

A string is an immutable sequence of bytes, conventionally UTF-8 encoded. Once created, a string cannot be modified. String concatenation creates a new string:

```
full := firstName + " " + lastName
```

Backtick-delimited strings are raw strings — no escape processing occurs, and they can span multiple lines. They are commonly used for regular expressions and SQL queries.

Strings can be indexed to access individual bytes:

```
s := "Hello"
first := s[0]  // 72, the byte value of 'H'
```

Indexing returns a `byte` (`uint8`), not a character. For Unicode text, use range iteration (covered in document 14), which yields runes.

Converting between `string` and `[]byte` always allocates a copy — there is no zero-copy path:

```
s := "hello"
b := []byte(s)    // allocates a new byte slice
s2 := string(b)   // allocates a new string
```

Modifying the byte slice does not affect the original string, because the copy is independent.

06 — Constants

Constants provide compile-time values that cannot be changed. Go distinguishes between untyped constants, which adapt to context, and typed constants, which enforce a specific type.

Declaration

```
const Pi = 3.14159265358979323846   // untyped constant
const Pi float64 = 3.14159265358979323846   // typed constant
```

A constant's value must be determinable at compile time. Function calls, variables, and runtime expressions cannot be used.

Untyped vs Typed Constants

Untyped constants carry a kind (integer, float, string, bool, rune) but no fixed type. They adapt to the context in which they are used:

```
const Pi = 3.14159265358979323846

var a float32 = Pi   // 3.1415927 (float32 precision)
var b float64 = Pi   // 3.141592653589793 (float64 precision)
var c int = Pi       // 3 (truncated to integer)
```

The untyped constant retains full precision until it is assigned to a typed variable. The compiler then rounds or truncates to fit the target type. A `float32` loses precision compared to `float64`. An `int` discards the fractional part entirely.

Typed constants behave like typed variables — they enforce their type strictly:

```
const Pi float64 = 3.14159265358979323846

var a float32 = Pi   // compile error: cannot use float64 constant as float32
var b float64 = Pi   // OK
```

The untyped form is more flexible. The typed form prevents accidental type mismatches. Use typed constants when the type matters to the API contract.

Grouped Constants

Multiple constants are grouped in a `const` block:

```
const (
    Monday    = 1
    Tuesday   = 2
```

```
    Wednesday = 3
)
```

Within a grouped block, lines that omit the `= expression` reuse the expression from the previous line. Lines that omit the type inherit the type from the previous line. A single block can mix multiple types — each new type declaration starts a fresh inheritance segment:

```
const (
    a int = 1    // type: int, value: 1
    b            // type: int (inherited), expression reused, value: 1
    c string = "hello"  // new type: string, value: "hello"
    d            // type: string (inherited), expression reused, value: "hello"
    e float64 = 3.14    // new type: float64, value: 3.14
    f            // type: float64 (inherited), expression reused, value: 3.14
)
```

Iota

`iota` is a counter that resets to 0 at each `const` declaration and increments by 1 for each constant within that declaration:

```
const (
    Sunday    = iota  // 0
    Monday            // 1
    Tuesday           // 2
    Wednesday         // 3
    Thursday          // 4
    Friday            // 5
    Saturday          // 6
)
```

Each `const` keyword starts a new declaration. Separate declarations reset `iota` independently:

```
const Sunday = iota   // 0 (first declaration)
const Monday = iota   // 0 (second declaration, iota resets)
```

Typed Iota Enumerations

`type Direction int` creates a new distinct type based on `int`. It behaves like `int` for arithmetic and storage, but the compiler treats it as a separate type — a plain `int` cannot be assigned to it without an explicit conversion. Use this pattern to type-safe `iota` constants:

```go
type Direction int

const (
    North Direction = iota
    East
    South
    West
)

func turn(d Direction) {
    // only accepts Direction, not plain int
}

turn(North)   // OK
turn(0)       // compile error: cannot use int as Direction
```

Iota Expressions

`iota` participates in expressions, incrementing normally within them:

```go
const (
    _  = iota      // 0, skipped with blank identifier
    KB = 1 << (10 * iota)  // 1024
    MB = 1 << (10 * iota)  // 1048576
    GB = 1 << (10 * iota)  // 1073741824
)
```

A common pattern for bitmask flags:

```go
const (
    Read    = 1 << iota  // 1
    Write                // 2
    Execute              // 4
)

permissions := Read | Write   // 3
```

Iota Increments Every Line, Even When Unused

`iota` is a per-line counter. It increments regardless of whether it is used on a given line:

```
const (
    A = iota   // 0
    B = iota   // 1
    C = 999    // 999 (iota is 2 here, unused)
    D = iota   // 3, not 2
)
```

Breaking an Iota Chain With a Different Type

When a line omits `= expression`, it reuses the type and expression from the previous line. If the previous line has a different type, `iota` produces a compile error:

```
const (
    A = iota     // 0
    B = iota     // 1
    C = "hello"  // untyped string
    D = iota     // compile error: cannot use untyped int 3 as string
)
```

`D` inherits `C`'s type (string) but `iota` produces an untyped int. Writing `= iota` explicitly on every line avoids this.

Untyped Predeclared Constants

Three constants are available without declaration:

```
true    // untyped bool
false   // untyped bool
nil     // untyped nil (for pointers, slices, maps, channels, functions, interfaces)
```

These are untyped and adapt to context. `nil` is the zero value for reference types and interfaces.

07 — Blank Identifier

The underscore `_` is a predeclared identifier that discards values. It is a write-only variable — you can assign to it but cannot read from it.

Discarding Return Values

Functions that return multiple values often require the caller to handle all of them. The blank identifier discards unwanted returns:

```
result, err := strconv.Atoi("42")
// use result, handle err

result, _ = strconv.Atoi("not-a-number")
// result is 0, error is discarded
```

Discarding Loop Index

`for range` returns both an index and a value. Use `_` to ignore the index:

```
names := []string{"Alice", "Bob", "Carol"}
for _, name := range names {
    fmt.Println(name)
}
```

Conversely, to ignore the value:

```
for i, _ := range names {
    fmt.Println(i)  // 0, 1, 2
}
```

Blank Imports

An import with `_` imports a package for its side effects only (typically its `init` function), without using any of its exported names:

```
import _ "database/sql/driver"
```

This pattern is used for driver registration — the package's `init` function registers itself with a framework, and the importing code never calls the package directly.

Type Assertions

When checking a type without extracting the value, discard the extracted value with `_`:

```go
var value interface{} = "hello"
_, ok := value.(int)
if !ok {
    fmt.Println("not an int")
}
```

Multiple Assignments

Multiple assignments to _ in the same scope do not conflict — _ always refers to the same discard slot:

```go
_, _ = func1()
_, _ = func2()   // no conflict, both discard to the same slot
```

This differs from languages where _ is a conventional variable name. In Go, _ is a language-level discard mechanism, not a variable.

08 — Operators

Go provides arithmetic, comparison, logical, bitwise, and assignment operators. Operator precedence follows standard C-like rules.

Arithmetic

```
sum := 10 + 3       // 13
diff := 10 - 3      // 7
product := 10 * 3   // 30
quotient := 10 / 3  // 3 (integer division truncates toward zero)
remainder := 10 % 3 // 1
```

Integer division truncates toward zero: `7 / 2` is `3`, `-7 / 2` is `-3`.

The `%` operator returns the remainder. The sign of the result matches the dividend, not the divisor:

```
10 % 3    // 1
-10 % 3   // -1
10 % -3   // 1
-10 % -3  // -1
```

Increment and Decrement

`++` and `--` are statements, not expressions. They cannot be used as a value:

```
i := 5
i++       // OK, statement
fmt.Println(i)  // 6

j := i++  // compile error: i++ is a statement, not an expression
```

This differs from C, Java, and C++, where `i++` returns the pre-increment value.

Comparison

```
5 == 5    // true
5 != 3    // true
5 < 10    // true
5 <= 5    // true
5 > 2     // true
5 >= 5    // true
```

Structs and arrays are comparable field-by-field:

```go
type Point struct {
    X, Y int
}

a := Point{1, 2}
b := Point{1, 2}
fmt.Println(a == b)  // true
```

Slices, maps, and functions are not comparable with `==` — it is a compile error. Use `reflect.DeepEqual` or `slices.Equal` instead.

Logical

```go
true && false   // false
true || false   // true
!true           // false
```

Logical operators use short-circuit evaluation: `false && expr` does not evaluate `expr`, and `true || expr` does not evaluate `expr`.

Bitwise

```go
a := 12  // 1100 in binary
b := 10  // 1010 in binary

a & b    // 8   (1000) - bitwise AND
a | b    // 14  (1110) - bitwise OR
a ^ b    // 6   (0110) - bitwise XOR
a &^ b   // 2   (0010) - bit clear (a AND NOT b)
a << 2   // 48  (110000) - left shift
a >> 1   // 6   (0110) - right shift
```

The `&^` (bit clear) operator is unique to Go. It clears the bits set in the right operand from the left operand:

```go
flags := Read | Write | Execute   // 7 (111)
flags = flags &^ Execute          // 3 (011) - Execute cleared
```

Assignment

```go
x = 5
x += 3    // x = x + 3
```

```
x -= 2     // x = x - 2
x *= 4     // x = x * 4
x /= 2     // x = x / 2
x %= 3     // x = x % 3
x &= 0xF   // x = x & 0xF
x |= 0x1   // x = x | 0x1
x ^= 0x5   // x = x ^ 0x5
x &^= 0x2  // x = x &^ 0x2
x <<= 1    // x = x << 1
x >>= 1    // x = x >> 1
```

Precedence

From highest to lowest (within each level, left-to-right):

Level	Operators	Description
1	`* / % << >> & &^`	Multiplicative, shift, bitwise AND, bit clear
2	`+ - | ^`	Additive, bitwise OR, XOR
3	`== != < <= > >=`	Comparison
4	`&&`	Logical AND
5	`||`	Logical OR

Parentheses override precedence. Unary `!`, `&`, and `*` bind tighter than all binary operators.

09 — Control Flow

Go provides conditionals, loops, and switches with minimal syntax. There are no parentheses around conditions, and braces are mandatory.

If

```go
if count > 0 {
    fmt.Println("positive")
}

if count > 0 {
    fmt.Println("positive")
} else {
    fmt.Println("zero or negative")
}
```

No parentheses around the condition. Braces are required — single-line if without braces is a compile error.

Init Statement in If

An init statement runs before the condition. The variables it declares are scoped to the if block:

```go
if count, err := getCount(); err == nil {
    fmt.Println(count)
}
// count and err are not accessible here
```

This pattern appears constantly in real Go code for error checks:

```go
if result, err := doSomething(); err != nil {
    log.Fatal(err)
}
// use result
```

For

`for` is the only loop construct in Go. There is no `while` or `do-while`.

C-Style For Loop

```go
for i := 0; i < 10; i++ {
    fmt.Println(i)
```

```
}
```

While Equivalent

Omit the init and post statements for a while loop:

```
i := 0
for i < 10 {
    fmt.Println(i)
    i++
}
```

Infinite Loop

Omit all clauses:

```
for {
    // runs forever until break
}
```

Range Loop

`for..range` iterates over arrays, slices, maps, strings, and channels. Covered in document 14.

Switch

```
switch day {
case "Monday":
    fmt.Println("start of week")
case "Friday":
    fmt.Println("end of week")
default:
    fmt.Println("midweek")
}
```

Cases do not fall through automatically. Each case terminates after its statements. Multiple values in one case use a comma:

```
switch grade {
case "A", "B":
    fmt.Println("pass")
case "C", "D":
    fmt.Println("marginal")
case "F":
```

```
    fmt.Println("fail")
}
```

Switch Without Value

A switch without a value works as an if-else chain:

```
switch {
case count > 10:
    fmt.Println("large")
case count > 0:
    fmt.Println("small")
default:
    fmt.Println("zero or negative")
}
```

Fallthrough

By default, each case terminates after its statements. `fallthrough` forces execution to continue into the next case:

```
switch grade {
case "B":
    fmt.Println("satisfactory")
    fallthrough
case "C":
    fmt.Println("needs improvement")
}
```

This prints both "satisfactory" and "needs improvement". The `fallthrough` keyword appears in real code (e.g., encoding packages) where multiple cases share cleanup logic. It transfers control to the next case's statements — the next case's condition is not evaluated.

Break and Continue

`break` exits the innermost loop or switch. `continue` skips to the next iteration:

```
for i := 0; i < 10; i++ {
    if i == 5 {
        break   // exits loop at i == 5
    }
    if i%2 == 0 {
        continue  // skips even numbers
```

```
    }
    fmt.Println(i)
}
```

Labeled Break

A labeled break exits a named outer loop:

```
outer:
for i := 0; i < 5; i++ {
    for j := 0; j < 5; j++ {
        if i+j == 4 {
            break outer
        }
    }
}
```

Goto

`goto` jumps to a labeled statement within the same function:

```
for i := 0; i < 100; i++ {
    if shouldExit(i) {
        goto cleanup
    }
}
cleanup:
fmt.Println("done")
```

`goto` cannot jump past variable declarations. It exists for deep nesting escape and is rarely used.

10 — Functions

Functions are the primary unit of code organization in Go. They support multiple return values, closures, variadic arguments, and deferred execution.

Declaration

```
func length(s string) int {
    return len(s)
}

result := length("Alice")  // 5
```

Multiple parameters of the same type share the type annotation:

```
func lessThan(a, b int) bool {
    return a < b
}
```

Multiple Return Values

Functions can return multiple values. The idiomatic pattern returns a result and an error:

```
func divide(a, b float64) (float64, error) {
    if b == 0 {
        return 0, fmt.Errorf("division by zero")
    }
    return a / b, nil
}

result, err := divide(10, 2)
if err != nil {
    log.Fatal(err)
}
fmt.Println(result)
```

Named return values are declared in the signature and returned by a bare `return`:

```
func divide(a, b float64) (quotient float64, err error) {
    if b == 0 {
        err = fmt.Errorf("division by zero")
        return
    }
    quotient = a / b
```

```
    return
}
```

Named returns reduce repetition but reduce clarity. Use them sparingly.

Functions as Values

Functions are first-class values — assignable to variables and passable as arguments:

```
type Selector func(int) bool

func isEven(n int) bool {
    return n%2 == 0
}

func selectAll(numbers []int, s Selector) []int {
    var result []int
    for _, n := range numbers {
        if s(n) {
            result = append(result, n)
        }
    }
    return result
}

evens := selectAll([]int{1, 2, 3, 4, 5, 6}, isEven)
```

Closures

A closure is a function literal that captures variables from its surrounding scope:

```
func makeCounter(start int) func() int {
    seed := 1000
    return func() int {
        start++
        return seed + start
    }
}

counter := makeCounter(0)
fmt.Println(counter())  // 1001
fmt.Println(counter())  // 1002
```

The closure captures both `start` (a parameter) and `seed` (a local variable). Both persist across calls.

Variadic Functions

A variadic function accepts a variable number of arguments of the same type. The variadic parameter must be the last parameter:

```
func sum(scale float64, values ...int) float64 {
    total := 0
    for _, v := range values {
        total += v
    }
    return float64(total) * scale
}

sum(1, 1, 2, 3)       // 6
sum(0.1, 1, 2, 3)     // 0.6
sum(10, 1, 2, 3, 4, 5) // 150
```

`values` is a slice inside the function. To pass a slice or array as variadic arguments, use the spread notation `...`:

```
slice := []int{1, 2, 3, 4, 5}
total := sum(1, slice...)  // 15

array := [3]int{1, 2, 3}
total := sum(1, array...)  // 6
```

Defer

`defer` schedules a function call to run when the surrounding function returns, regardless of how it returns. The defer statement can appear anywhere in the function, as long as it is reached before the return:

```
func process() {
    file, err := os.Open("data.txt")
    if err != nil {
        return
    }
    defer file.Close()

    // work with file
```

```
    // file.Close() runs when process() returns, even if there's an early return
}
```

Deferred calls execute in LIFO order — the last deferred call runs first:

```
for i := 0; i < 3; i++ {
    defer fmt.Print(i)
}
// prints: 2 1 0
```

Arguments to a deferred function are evaluated immediately at the `defer` statement, not when the deferred call runs:

```
count := 0
defer fmt.Println(count)  // will print 0, not the final value
count = 5
```

To capture the final value, defer a function literal:

```
count := 0
defer func() {
    fmt.Println(count)  // prints 5
}()
count = 5
```

11 — Panic and Recover

Panic stops normal execution and begins unwinding the stack. Recover catches a panic inside a deferred function and restores normal execution. These are for truly unexpected conditions, not for normal error handling.

Panic

`panic(value)` halts the current function, runs deferred calls, then unwinds to the caller, running its deferred calls, and so on up the stack:

```
func main() {
    defer fmt.Println("cleanup runs")
    panic("something went wrong")
    fmt.Println("this never runs")
}
// Output:
// cleanup runs
// panic: something went wrong
// [stack trace]
```

`panic()` accepts any value, conventionally a string or error. Without a `recover` anywhere in the call chain, the program terminates with a stack trace.

Recover

`recover()` inside a deferred function catches a panic and restores normal execution:

```
func safeCall() {
    defer func() {
        if r := recover(); r != nil {
            fmt.Printf("recovered from: %v\n", r)
        }
    }()

    panic("something went wrong")
    fmt.Println("this never runs")
}

func main() {
    safeCall()
    fmt.Println("execution continues")
}
```

```
// Output:
// recovered from: something went wrong
// execution continues
```

`recover()` returns `nil` when there is no active panic. It returns a non-`nil` value only when called inside a deferred function during an active panic. Calling `recover()` directly (not inside a deferred function) always returns `nil`.

When to Use Panic

Panic is for bugs — conditions that should be impossible if the code is correct. Not for "the world did something unexpected," but for "the code reached a state it shouldn't have." The distinction: can any caller in the chain do something useful with the failure? If yes, return an error. If no, because the condition means the code is wrong, panic.

```
// A state machine that should never reach this state
switch s.status {
case "pending":
    handlePending(s)
case "done":
    handleDone(s)
default:
    panic(fmt.Sprintf("unreachable state: %s", s.status))
}
```

```
// A function contract was violated - the caller passed something impossible
func parseToken(t Token) Value {
    if t.Kind == 0 {
        panic("parseToken called with zero Kind - caller bug")
    }
    // ...
}
```

Other examples: a map lookup that must succeed because the key was just inserted; a channel receive that must return a value because the sender is guaranteed to be running; a switch on an enumerated type that falls through all cases. These are not runtime failures — they are signals that a bug exists somewhere in the code.

When Not to Use Panic

Conditions where the caller has a meaningful response:

- User input is invalid — return an error; the caller can show a message

- A file doesn't exist — return an error; the caller can use a default
- A network request fails — return an error; the caller can retry
- A database query returns no rows — return an empty result

These are not bugs. They are the program interacting with an unpredictable world. Use the `(result, error)` pattern from document 18.

Where to Place Recover

Recover does not fix the bug. It contains it. A panic indicates something is wrong in the code — recover's job is to prevent that bug from crashing the entire system while logging enough information to find and fix it.

Place recover only at system boundaries, where you can cleanly terminate the current unit of work without affecting others:

- HTTP handlers — kill the request, return 500, keep the server alive
- Goroutine entry points — let one goroutine die without killing the program
- Test runners — let one test fail without stopping the suite

```
func handleRequest(w http.ResponseWriter, r *http.Request) {
    defer func() {
        if rec := recover(); rec != nil {
            http.Error(w, "internal server error", 500)
            log.Printf("panic in %s %s: %v", r.Method, r.URL, rec)
        }
    }()

    // request handling
}
```

Do not place recover in the middle of a call chain. It does not turn a bug into a handled error — it just hides where the bug is.

Runtime Panics

Panics from the runtime itself — nil pointer dereference, out-of-bounds slice access, map write on nil map — cannot be caught by `recover` in all Go versions. This is a factual limitation. Application-level panics (explicit `panic()` calls) are reliably recoverable.

12 — Arrays and Slices

Go provides arrays for fixed-size sequences and slices for variable-length views over underlying arrays. Slices are the everyday sequence type.

Arrays

An array has a fixed size that is part of its type. `[3]int` and `[4]int` are different types:

```
var arr [3]int            // [0 0 0]
arr = [3]int{1, 2, 3}     // [1 2 3]
arr[0] = 10               // [10 2 3]
```

You can also use index-keyed (sparse) literals to set specific positions, leaving the rest as zero values:

```
arr := [5]int{2: 10, 4: 20}  // [0 0 10 0 20]
```

Arrays are value types. Assigning or passing an array copies all elements:

```
a := [3]int{1, 2, 3}
b := a       // full copy
b[0] = 99
fmt.Println(a)  // [1 2 3], unchanged
```

Arrays are rarely used directly. Their fixed size and copy semantics make them impractical for most use cases. They appear as the underlying storage for slices and in fixed-size data structures.

Slices Are Views Over Arrays

A slice is a view over an underlying array. It does not own the data — it references a portion of an array with a starting index, a length, and a capacity. The slicing operator `arr[low:high]` creates a slice from an array:

```
arr := [5]int{0, 1, 2, 3, 4}
s := arr[1:4]    // view over arr[1], arr[2], arr[3]
fmt.Println(s)   // [1 2 3]
```

Slice type syntax omits the size: `[]int` rather than `[5]int` which defines an array.

Omit `low` to start from 0, omit `high` to go to the end:

```
arr[:3]   // [0 1 2]
arr[2:]   // [2 3 4]
arr[:]    // [0 1 2 3 4]
```

Shared Underlying Array

A slice shares the underlying array. Mutations go both ways — changing the slice affects the array, and changing the array affects the slice:

```
arr := [5]int{0, 1, 2, 3, 4}
s := arr[1:3]    // [1 2]

s[0] = 99
fmt.Println(arr)  // [0 99 2 3 4], slice modified the array

arr[2] = 77
fmt.Println(s)    // [99 77], array modified the slice
```

Slicing a Slice

The same slicing syntax works on slices, producing another view over the same backing array:

```
s := []int{0, 1, 2, 3, 4}
sub := s[1:4]    // [1 2 3]
sub2 := sub[1:3] // [2 3] - still views the same backing array
```

The shared-array behavior compounds: all three slices reference the same underlying data. A mutation through any of them is visible to the others.

Slice Literals

A slice literal `[]int{1, 2, 3, 4, 5}` is shorthand that creates a backing array behind the scenes and returns a slice view over it. The array is not accessible — only the slice is:

```
s := []int{1, 2, 3, 4, 5}
```

This is the most common way to create a slice. The view semantics are the same as slicing an array — the literal just hides the intermediate step.

Make

`make` creates a slice with an explicit backing array of a given length and capacity. All elements are initialized to their zero value (0 for numbers, false for bools, "" for strings, nil for pointers and interfaces):

```
s := make([]int, 5)          // [0 0 0 0 0], length 5, capacity 5
s := make([]int, 3, 10)      // [0 0 0], length 3, capacity 10
```

- **Length** is the number of elements accessible through the slice.

- **Capacity** is the total number of elements in the backing array from the slice's start — the available space for growth.

Len and Cap

`len` and `cap` are language built-ins. With an array, both return the array's size:

```
arr := [5]int{0, 1, 2, 3, 4}
len(arr)  // 5
cap(arr)  // 5
```

With a slice, they return the length and capacity:

```
s := make([]int, 3, 10)
len(s)  // 3 - number of accessible elements
cap(s)  // 10 - total space in the backing array from the slice's start
```

For a sub-slice, capacity reflects the remaining room in the backing array:

```
arr := [5]int{0, 1, 2, 3, 4}
s := arr[1:3]
len(s)  // 2 - elements 1 and 2
cap(s)  // 4 - elements 1 through 4 are reachable from the backing array
```

Append

`append` adds elements to a slice and returns the updated slice:

```
s := []int{1, 2}
s = append(s, 3)       // [1 2 3]
s = append(s, 4, 5)    // [1 2 3 4 5]
s = append(s, []int{6, 7}...)  // [1 2 3 4 5 6 7]
```

The original variable must be reassigned — `append` may allocate a new backing array when the slice has no remaining capacity (length equals capacity):

```
s := []int{1, 2}
append(s, 3)    // s is still [1 2], the result was discarded
s = append(s, 3)  // s is now [1 2 3]
```

Appending to a Sub-Slice When a sub-slice shares a backing array with the original and has spare capacity, `append` writes into that shared space — silently overwriting elements the original slice can see:

```
s := []int{0, 1, 2, 3, 4}
sub := s[1:3]       // [1 2], capacity 4 (elements 1-4 of backing array)
sub = append(sub, 9)  // writes 9 into backing array at index 3
fmt.Println(s)       // [0 1 2 9 4], original silently modified
```

The append did not allocate a new array because the backing array had room. It wrote into the shared space, overwriting `s[3]`.

To prevent this, use the full slice expression `s[low:high:max]` to limit the sub-slice's capacity. Setting `max` equal to `high` leaves no room for growth, forcing `append` to allocate a new backing array:

```
s := []int{0, 1, 2, 3, 4}
sub := s[1:3:3]      // [1 2], length 2, capacity 2 - no spare room
sub = append(sub, 9)  // must allocate new backing array
fmt.Println(s)       // [0 1 2 3 4], original unchanged
```

The third index (`max`) sets the capacity of the new slice to `max - low`. When capacity equals length, `append` has no choice but to grow into a new array.

Copy

`copy` copies elements from a source slice to a destination slice:

```
dst := make([]int, 3)
src := []int{1, 2, 3, 4, 5}
n := copy(dst, src)   // copies 3 elements, n == 3
fmt.Println(dst)      // [1 2 3]
```

`copy` handles overlapping slices correctly. It is also the standard way to detach a sub-slice from its backing array by creating a new slice and copying into it:

```
s := []int{0, 1, 2, 3, 4}
attached := s[1:3]              // [1 2], shares backing array with s
detached := make([]int, len(attached))  // same length, all elements zero-valued
copy(detached, attached)                // fills detached with the values
detached[0] = 99
fmt.Println(s)                  // [0 1 2 3 4], original unchanged
```

Nil Slice vs Empty Slice

```
var s []int          // nil slice
s == nil             // true
len(s)               // 0
```

```
e := []int{}          // empty slice
e == nil              // false
len(e)                // 0
```

- `var s []int` declares a slice without a literal value, so it takes the zero value of its type — which for slices is `nil`.
- `[]int{}` explicitly creates an empty slice with its own empty backing array.

Both have length 0 and behave identically in most operations. The difference matters when serializing to JSON: `json.Marshal` produces `null` for a nil slice and `[]` for an empty slice.

String to Byte or Rune Slice

A string can be converted directly to `[]byte` (bytes) or `[]rune` (Unicode code points). Both conversions always allocate a copy — there is no zero-copy path. Modifying the slice does not affect the original string.

`[]byte(s)` iterates the string byte by byte:

```
s := "hello"
b := []byte(s)    // [104 101 108 108 111]
```

`[]rune(s)` decodes the string as UTF-8 and produces one element per Unicode code point:

```
s := "cafe\u0301"       // e with combining accent, 5 bytes
r := []rune(s)          // [99 97 102 101 769], 5 runes
```

These are the only two direct conversions from string to slice. Converting to other types (e.g., integers) requires the `strconv` package.

Slice Comparison

Slices cannot be compared with `==` — it is a compile error. Two slices may reference different backing arrays with identical contents, and Go has no defined semantics for value equality of slices:

```
a := []int{1, 2}
b := []int{1, 2}
a == b  // compile error: slice can only be compared to nil
```

Use `slices.Equal` from the `slices` package (Go 1.21+):

```
import "slices"
slices.Equal(a, b)  // true
```

Or `reflect.DeepEqual` for older Go versions. Note that `reflect.DeepEqual` is significantly slower and has different semantics for nested types.

Multidimensional Slices

A slice of slices — `[][]int` — represents a grid or matrix. Each row is an independent slice, so rows can have different lengths (a **jagged array**):

```
grid := [][]int{
    {1, 2, 3},
    {4, 5},
    {6, 7, 8, 9},
}
fmt.Println(grid[0][2])  // 3 - row 0, column 2
fmt.Println(grid[2][3])  // 9 - row 2, column 3
```

Access elements with chained indexing: `grid[row][col]`. Iterate with nested loops:

```
for i, row := range grid {
    for j, val := range row {
        fmt.Printf("grid[%d][%d] = %d\n", i, j, val)
    }
}
```

To create a rectangular grid without a literal, allocate the outer slice then each row:

```
func makeGrid(rows, cols int) [][]int {
    grid := make([][]int, rows)
    for i := range grid {
        grid[i] = make([]int, cols)
    }
    return grid
}
```

The same pattern extends to three or more dimensions (`[][][]int`, etc.), though beyond two dimensions the code becomes unwieldy and a flat slice with manual index arithmetic is often clearer.

13 — Maps

A map is Go's built-in key-value type. Keys must be comparable (support `==`) — see the Comparable Keys section below. Values can be any type.

Declaration and Creation

```
var scores map[string]int    // nil map
scores = make(map[string]int) // usable map
```

A nil map cannot be written to — it panics. It does read, returning the zero value for any key:

```
var m map[string]int
fmt.Println(m["key"])  // 0, does not panic
m["key"] = 1           // panic: assignment to entry in nil map
```

Initialize with literals:

```
scores := map[string]int{
    "Alice": 95,
    "Bob":   87,
}
```

Setting and Getting

```
scores["Alice"] = 95

value := scores["Alice"]       // 95
value := scores["Missing"]     // 0, silent zero value
```

The two-value form checks for key existence:

```
value, ok := scores["Alice"]
if ok {
    fmt.Printf("Alice scored %d\n", value)
} else {
    fmt.Println("Alice not found")
}
```

Without the `ok` check, a missing key returns the zero value silently — a source of subtle bugs. The two-value form is the standard approach.

Deleting

```
delete(scores, "Bob")
```

`delete` removes the key. It does nothing if the key is absent — no error, no panic.

Iteration

```
for name, score := range scores {
    fmt.Printf("%s: %d\n", name, score)
}
```

Iteration order is intentionally random on every run. Do not write code that depends on map iteration order.

Maps Are Reference Types

Passing a map to a function shares the same underlying map:

```
func updateScores(m map[string]int) {
    m["Charlie"] = 92
}

scores := map[string]int{"Alice": 95}
updateScores(scores)
fmt.Println(scores)  // map[Alice:95 Charlie:92]
```

Reassigning the map variable inside the function does not affect the caller:

```
func replaceScores(m map[string]int) {
    m = map[string]int{"New": 100}  // only changes the local variable
}
```

Comparable Keys

Map keys must be **comparable** — a compile-time type property meaning the type supports `==` and `!=`. This is not a runtime check; the compiler rejects a map declaration if the key type is not comparable.

Primitive Comparable Types

These built-in types are comparable:

- Integers: `int`, `int8`, `int16`, `int32`, `int64`, `uint`, `uint8`, `uint16`, `uint32`, `uint64`, `uintptr`
- Floats: `float32`, `float64`

- Strings
- Booleans
- Pointers (of any type)
- `complex64`, `complex128`

Composite Comparable Types

Structs and arrays are comparable **recursively** — the type is comparable only if every component is comparable:

```
type Config struct {
    Host string
    Port int
}

// OK - all fields are comparable
m := map[Config]bool{
    {Host: "localhost", Port: 8080}: true,
}
```

An array is comparable if its element type is comparable:

```
// OK - [3]int is comparable because int is comparable
m := map[[3]int]string{
    {1, 2, 3}: "triple",
}
```

Nested structs follow the same rule — every field at every level must be comparable:

```
type Address struct {
    Street string
    City   string
}

type Person struct {
    Name    string
    Address Address  // OK - Address is comparable (all fields are strings)
}

// OK - Person is comparable
m := map[Person]int{}
```

Non-Comparable Types

Slices, maps, and functions are **never** comparable. They are reference types — two variables may point to different underlying data with the same contents, and there is no defined semantics for value equality:

```
map[[]int]string{}    // compile error: []int is not comparable
map[map[int]string]string{} // compile error: map[int]string is not comparable
map[func()]string{}   // compile error: func() is not comparable
```

Recursive Non-Comparability

A struct containing any non-comparable field is itself non-comparable, even if all other fields are comparable:

```
type BadKey struct {
    ID   int
    Tags []string  // slice makes the entire struct non-comparable
}

map[BadKey]int{}  // compile error: BadKey is not comparable
```

This rule applies at any nesting depth. A struct that contains another struct that contains a slice is also non-comparable.

Workaround: Use a Comparable Surrogate

When you need to index by a struct with non-comparable fields, derive a comparable key:

```
type Item struct {
    ID   int
    Tags []string
}

// Use the ID as the map key instead
items := map[int]*Item{}
items[item.ID] = &item
```

Or use a string representation of the fields you need to match on.

14 — Range

`for..range` iterates over collections, yielding both an index and a value on each iteration. It is the primary way to traverse arrays, slices, maps, and strings.

Arrays and Slices

Range over an array or slice yields the index and a copy of each element:

```
names := []string{"Alice", "Bob", "Carol"}
for i, name := range names {
    fmt.Println(i, name)  // 0 Alice, 1 Bob, 2 Carol
}
```

Use the blank identifier to discard the index or value:

```
for _, name := range names {
    fmt.Println(name)  // value only
}

for i := range names {
    fmt.Println(i)  // index only
}
```

Range Creates Copies

The value returned by range is a copy of the element, not a reference. Mutating it does not mutate the underlying collection:

```
scores := []int{1, 2, 3}
for i, score := range scores {
    score = score * 10  // modifies the copy, not the slice
}
fmt.Println(scores)  // [1 2 3], unchanged
```

To mutate elements, use the index. Omit the value variable to get only the index:

```
for i := range scores {
    scores[i] = scores[i] * 10
}
fmt.Println(scores)  // [10 20 30]
```

Maps

Range over a map yields key-value pairs. Iteration order is intentionally random on every run — do not write code that depends on a specific order:

```
ages := map[string]int{"Alice": 30, "Bob": 25}
for name, age := range ages {
    fmt.Printf("%s is %d\n", name, age)  // order is random
}
```

Strings

Range over a string yields runes (Unicode code points), not bytes. The index is the byte offset, not the rune position:

```
for i, r := range "caf\u00e9 del mar" {
    fmt.Printf("%d %q\n", i, r)
    // 0 'c'
    // 1 'a'
    // 2 'f'
    // 3 'é'    (2 bytes in UTF-8)
    // 5 ' '    - byte offset jumped from 3 to 5
    // 6 'd'
    // 7 'e'
    // 8 'l'
    // ...
}
```

The precomposed é (U+00E9) occupies 2 bytes. After it, every subsequent byte offset is shifted by 1 compared to the rune position.

If the string contains invalid UTF-8, range substitutes the bad byte with the Unicode replacement character `U+FFFD` and continues — it does not panic:

```
s := "hello\x80world"  // \x80 is invalid UTF-8
for _, r := range s {
    fmt.Printf("%c ", r)
}
// Output: h e l l o \textquestiondown w o r l d
```

To detect invalid UTF-8 rather than silently replacing it, use the `unicode/utf8` package.

Range over a channel receives values until the channel is closed. Covered in document 22.

15 — Structs

A struct groups named fields into a composite type.

Definition

```
type Person struct {
    Name string
    Age  int
}
```

Field names that start with an uppercase letter are exported. Lowercase field names are unexported and accessible only within the same package.

Initialization

Named fields are the preferred form — they are order-independent and self-documenting:

```
p := Person{
    Name: "Alice",
    Age:  30,
}
```

Positional initialization also works but is fragile to field reordering:

```
p := Person{"Alice", 30}
```

Field Access

```
p := Person{Name: "Alice", Age: 30}
fmt.Println(p.Name)  // Alice
p.Age = 31
```

Anonymous Structs

Structs can be defined inline without a named type, the first part is the *type definition* and the second part is the *literal* that defines the actual values:

```
config := struct {
    Host string
    Port int
}{
    Host: "localhost",
```

```
    Port: 8080,
}
```

Anonymous structs are used for ad-hoc data, JSON payloads, and test fixtures. Two anonymous struct types are identical only if they have exactly the same fields in the same order.

Methods

A method is a function bound to a type:

```
func (p Person) Greet() string {
    return fmt.Sprintf("Hi, I'm %s", p.Name)
}

p := Person{Name: "Alice", Age: 30}
fmt.Println(p.Greet())  // Hi, I'm Alice
```

Receivers

A method uses a type as its receiver. A value receiver (`(p Person)`) gets a copy; a pointer receiver (`(p *Person)`) gets the address and can modify the original. The full explanation of value vs pointer receivers, including when to use each, is covered in document 16.

Struct Embedding

A type included without a field name is embedded — its fields and methods are promoted to the outer struct:

```
type Address struct {
    City    string
    Country string
}

func (a Address) Location() string {
    return fmt.Sprintf("%s, %s", a.City, a.Country)
}

type Person struct {
    Name    string
    Age     int
    Address // embedded
}
```

```
p := Person{
    Name:    "Alice",
    Age:     30,
    City:    "London",
    Country: "UK",
}
```

You can also initialize using the embedded type name explicitly, or mix both approaches:

```
p := Person{
    Name:    "Alice",
    Age:     30,
    Address: Address{City: "London", Country: "UK"},
}
```

The initialization form has no effect on field access. Regardless of how the struct was created, promoted fields can always be accessed directly or through the embedded type name, and both forms are interchangeable anywhere in the code:

```
p.City = "Milan"           // direct access
fmt.Println(p.Address.City) // explicit path, equivalent
```

Embedding is Go's primary composition mechanism. It is not inheritance — the embedded type is not a subtype of the outer struct.

Promoted Field Conflicts

When two embedded types (or an embedded type and the outer struct) share a field or method name, the promotion is ambiguous. The compiler rejects the unqualified name — you must use the explicit path through the embedded type:

```
type Contact struct {
    Email string
}

type Account struct {
    Email string
}

type User struct {
    Name    string
    Contact // embedded
    Account // embedded
```

```
}

u := User{}
u.Email       // compile error: ambiguous selector u.Email
u.Contact.Email = "a@b.c"  // OK - explicit path
u.Account.Email = "x@y.z"  // OK - explicit path
```

The same rule applies if an embedded type has a field with the same name as a field declared directly on the outer struct. The outer struct's field takes priority for the unqualified name; the embedded field is still accessible through its type name.

Struct Tags

Tags are metadata attached to fields as backtick-delimited strings, read via reflection (see document 27):

```
type User struct {
    Name    string `json:"name"`
    Age     int    `json:"age,omitempty"`
    Email   string `json:"email" db:"user_email"`
    Password string `json:"-"`
}
```

Common tag keys include `json`, `xml`, `yaml`, and `db`. Multiple key-value pairs are space-separated within a tag. The `omitempty` option omits the field during serialization if it has the zero value. The `-` value skips the field entirely.

Tags are conventions, not enforced by the compiler. A package that reads tags defines its own format.

16 — Pointers

Pointers allow functions to modify the caller's variables and avoid copying large values. Go pointers are restricted compared to C — there is no pointer arithmetic, and the runtime manages memory automatically.

Pointer Basics

`*T` is a pointer to a value of type `T`. The `&` operator takes the address of a variable:

```
count := 5
ptr := &count   // ptr is *int, points to count
```

Dereference with `*` to read or write the value at the address:

```
*ptr = 10       // modifies the value count points to
fmt.Println(count)  // 10
```

A pointer variable initialized without a value is `nil`:

```
var ptr *int    // nil
if ptr != nil {
    fmt.Println(*ptr)
}
```

Dereferencing a `nil` pointer causes a runtime panic.

No Pointer Arithmetic

Go does not support pointer arithmetic. You cannot increment a pointer, subtract pointers, or use pointers to walk through memory:

```
ptr++       // compile error
ptr + 1     // compile error
```

This restriction eliminates an entire class of memory safety bugs. For low-level operations, the `unsafe` package (document 28) provides limited capabilities.

Passing by Value vs Passing by Pointer

Go passes all arguments by value. A function receives a copy of the argument:

```
func increment(x int) {
    x++   // modifies the copy, not the original
}

count := 5
```

```
increment(count)
fmt.Println(count)  // 5, unchanged
```

Passing a pointer allows the function to modify the caller's variable:

```
func increment(x *int) {
    *x++   // modifies the value at the address
}

count := 5
increment(&count)
fmt.Println(count)  // 6
```

Use value passing when the function does not need to modify the argument. Use pointer passing when modification is needed or when copying the value is expensive (e.g., large structs).

Automatic Pointer Dereference for Field Access

When you have a pointer to a struct, Go automatically dereferences it for field access. There is no `->` operator — the dot syntax works for both values and pointers:

```
type Person struct {
    Name string
    Age  int
}

person := Person{Name: "Alice", Age: 30}
fmt.Println(person.Name)    // Alice - value access

ptr := &person
fmt.Println((*ptr).Name)    // Alice - explicit dereference
fmt.Println(ptr.Name)       // Alice - equivalent, automatic dereference
```

The explicit dereference `(*ptr).Name` is valid but unnecessary — `ptr.Name` is the idiomatic form.

Pointer Receivers

A method with a pointer receiver can mutate the struct and avoids copying it:

```
type Counter struct {
    count int
}

func (c *Counter) Increment() {
```

```
    c.count++
}

counter := Counter{count: 0}
counter.Increment()   // Go automatically takes the address: (&counter).Increment()
fmt.Println(counter.count)  // 1
```

Go handles the address-taking automatically when calling a pointer method on a value. The reverse is also true — calling a value method on a pointer automatically dereferences:

```
func (c Counter) Value() int {
    return c.count
}

ptr := &Counter{count: 5}
fmt.Println(ptr.Value())  // 5, Go automatically dereferences
```

Pointers with Structs, Slices, and Maps

Structs

Structs are value types — assigning or passing a struct copies all fields. Use `&` to get a pointer to a struct:

```
p := &Person{Name: "Alice", Age: 30}
```

Slices and Maps

Slices and maps are special types that carry a pointer to data allocated on the heap. A slice pointer is called header and it contains, other than the pointer itself, extra fields (length and capacity). A map variable is simply a pointer to a heap-allocated map structure. In both cases, assigning or passing the variable *copies* the pointer (or the header), *not* the underlying data.

For slices, the header fields can differ between copies pointing to the same array — one view might have length 3 and another length 5 over the same backing data. This is the basis of slice views. For maps, there is no such distinction: two variables pointing to the same map are fully equivalent, as they are just plain pointers.

When a slice or map is passed to a function, the pointer is copied but the underlying data is shared. Modifying the data inside the function changes what the caller sees. For slices, modifying the header (e.g., via `append`) only affects the copy — the caller's header stays unchanged.

```
original := []int{1, 2, 3}
copy := original
```

```
copy[0] = 99
fmt.Println(original[0])  // 99 - shared underlying array
```

The same applies to maps:

```
m := map[string]int{"a": 1}
n := m
n["a"] = 99
fmt.Println(m["a"])  // 99 - shared underlying map
```

Because they already behave like pointers, taking a pointer to a slice or map (`*[]int`, `*map[string]int`) is rare. It is only needed when you need to reassign the variable itself (e.g., setting it to `nil` or replacing it with a different slice/map) from within a function.

Memory Allocation

Go decides automatically whether a variable lives on the stack or the heap. The programmer does not control this decision. There is no `new` operator equivalent to C's `malloc` — the `new` built-in allocates zeroed memory on the heap and returns a pointer, but it is rarely used in practice:

```
ptr := new(int)   // allocates zeroed int, returns *int
*ptr = 10
```

Prefer composite literals with `&` over `new`:

```
ptr := &Counter{count: 0}   // clearer intent
```

Stack vs Heap

Go allocates variables on either the **stack** or the **heap**. Stack allocation is fast — memory is claimed and released by moving a pointer. Heap allocation is slower and requires the garbage collector to reclaim memory.

The compiler decides where a variable lives through **escape analysis**. If the compiler can prove a variable's lifetime is confined to the current function, it goes on the stack and is freed automatically when the function returns. If the variable might outlive the function — for example, a pointer returned to the caller, or a reference stored in a global — it *escapes* to the heap:

```
func onStack() int {
    x := 5
    return x   // x is a simple value, stays on stack
}

func escapesToHeap() *int {
    x := 5
```

```
    return &x  // x must survive the function, moves to heap
}
```

Variables on the stack are cleaned up automatically when the function returns. Heap-allocated variables are reclaimed by the **garbage collector** — a concurrent algorithm that runs most of its work alongside your program but still requires brief stop-the-world pauses (typically sub-millisecond) at key points. Under heavy allocation pressure or large heaps, these pauses can grow noticeably. Understanding escape analysis helps predict allocation patterns and avoid unnecessary heap pressure.

You can inspect escape analysis with `go build -gcflags="-m"`.

17 — Interfaces

An interface defines a set of method signatures. Any type that implements all methods satisfies the interface — there is no declaration, no `implements` keyword, and no explicit relationship.

Definition

```
type Stringer interface {
    String() string
}
```

Implicit Satisfaction

A type satisfies an interface by implementing its methods. The compiler verifies satisfaction at the point of assignment:

```
type Circle struct {
    Radius float64
}

func (c Circle) String() string {
    return fmt.Sprintf("circle radius %.2f", c.Radius)
}

var s Stringer = Circle{Radius: 5}  // Circle satisfies Stringer
fmt.Println(s.String())             // circle radius 5.00
```

No declaration links `Circle` to `Stringer`. The match is purely structural.

Method Sets and Receiver Types

A type `T` and its pointer `*T` have different method sets. `*T` includes methods defined with both value and pointer receivers — Go automatically dereferences the pointer when calling a value-receiver method (`(*t).ValueMethod()` becomes `t.ValueMethod()`). `T` only includes methods defined with value receivers, because Go cannot automatically take the address of a non-addressable value.

This matters when satisfying interfaces. If a type mixes receiver types, only `*T` satisfies an interface requiring both:

```
type T struct{}

func (t T) ValueMethod() {}
func (t *T) PointerMethod() {}
```

```
var i interface{ ValueMethod(); PointerMethod() }
i = &T{}  // OK - *T has both methods
i = T{}   // compile error: T does not have PointerMethod
```

The same problem appears when passing values to functions that expect an interface:

```
func process(i interface{ ValueMethod(); PointerMethod() }) {}

var t T
process(t)   // compile error
process(&t)  // OK
```

Fix: pick one receiver type per struct and use it consistently across all methods. If any method needs to mutate the receiver, use pointer receivers for all methods on that type.

Small Interfaces

Interfaces are typically small — one or two methods is idiomatic. The standard library is built on small interfaces like `io.Reader` and `io.Writer`:

```
type Reader interface {
    Read(p []byte) (n int, err error)
}

type Writer interface {
    Write(p []byte) (n int, err error)
}
```

Interface Embedding

One interface can include another, combining their method sets:

```
type ReadWriter interface {
    Reader
    Writer
}
```

A type that satisfies `ReadWriter` implements both `Read` and `Write`.

The Empty Interface

`any` is an alias for `interface{}` — it accepts any value:

```
var value any = 42
value = "hello"
value = Circle{Radius: 5}
```

Once a value is `any`, the compiler cannot check how it is used. Type information is available only at runtime through type assertions. Use `any` as a last resort.

Type Assertion

Extract the concrete type from an interface value:

```
var s Stringer = Circle{Radius: 5}
c := s.(Circle)        // c is of type Circle
fmt.Println(c.Radius)  // 5
```

A wrong assertion panics. The safe form returns a boolean:

```
c, ok := s.(Circle)
if ok {
    fmt.Println(c.Radius)
}
```

Type Switch

Dispatch on the concrete type stored in an interface:

```
func describe(value any) string {
    switch v := value.(type) {
    case int:
        return fmt.Sprintf("int: %d", v)
    case string:
        return fmt.Sprintf("string: %s", v)
    case Circle:
        return fmt.Sprintf("circle: %.2f", v.Radius)
    default:
        return "unknown type"
    }
}
```

The type switch variable `v` has the concrete type within each case.

Nil Pointer Assigned to Interface Is Not Nil

An interface value is a two-field header: a **pointer** to the underlying data and a **type** describing it. The interface is nil only when **both** fields are nil. Assigning a nil pointer sets the type field, making

the interface non-nil:

```
var ptr *int = nil
var i interface{} = ptr
fmt.Println(i == nil)  // false - type is *int, pointer is nil
```

The same with `any` (the common alias for `interface{}`):

```
var ptr *int = nil
var i any = ptr
fmt.Println(i == nil)  // false
```

This is a frequent source of bugs when returning errors as interfaces (remember that `error` is an interface):

```
func f() error {
    var err *os.PathError = nil
    // ... some logic that might set err ...
    return err  // returns non-nil interface even when err is nil
}
```

Return the concrete type directly, or explicitly return `nil`:

```
if err != nil {
    return err
}
return nil
```

18 — Error Handling

Go signals errors through return values, not exceptions. Errors are ordinary values — there is no exception handling, no stack unwinding, and no try-catch. Control flow remains explicit and linear. The `error` type is a built-in interface, and the convention is to return errors as the last value from functions that may fail.

The Error Interface

```
type error interface {
    Error() string
}
```

Any type with an `Error() string` method satisfies the `error` interface.

Creating Errors

`errors.New` creates a simple error from a message:

```
import "errors"

func validateAge(age int) error {
    if age < 0 {
        return errors.New("age cannot be negative")
    }
    if age > 150 {
        return errors.New("age is unrealistic")
    }
    return nil
}
```

Checking Errors

The standard pattern checks the error immediately after the call:

```
err := validateAge(-5)
if err != nil {
    fmt.Println(err)  // prints: age cannot be negative
}
```

With a result value:

```
result, err := parseInput(data)
if err != nil {
```

```
    return err
}
// use result
```

Sentinel Errors

Package-level error values that functions return and callers can check with `==`:

```
var ErrNotFound = errors.New("not found")

func lookup(key string) (string, error) {
    value, ok := database[key]
    if !ok {
        return "", ErrNotFound
    }
    return value, nil
}

_, err := lookup("missing")
if err == ErrNotFound {
    fmt.Println("key not in database")
}
```

Sentinel errors are exported so callers can check for them. They are compared by identity, not by message text.

Chaining Errors

Errors can be chained by implementing `Unwrap() error`, which returns the underlying error:

```
type wrappedError struct {
    msg string
    err error
}

func (e *wrappedError) Error() string {
    return e.msg + ": " + e.err.Error()
}

func (e *wrappedError) Unwrap() error {
    return e.err
}
```

`Unwrap` is the mechanism that `errors.Is` and `errors.As` use to traverse a chain of errors. They call `Unwrap` repeatedly until it returns `nil`.

Wrapping Errors

`fmt.Errorf` with `%w` wraps an error with additional context:

```
func loadConfig(path string) error {
    data, err := os.ReadFile(path)
    if err != nil {
        return fmt.Errorf("load config: %w", err)
    }
    // parse data...
    return nil
}
```

Wrapping adds context for humans while preserving the original error for programmatic checks. The call chain builds a readable message: `load config: open config.yaml: no such file or directory`.

Unwrapping Errors

`errors.Is` checks if an error anywhere in the chain matches a target value:

```
data, err := os.ReadFile("missing.txt")
if errors.Is(err, os.ErrNotExist) {
    fmt.Println("file does not exist")
}
```

Unlike `==`, which only matches the exact error value, `errors.Is` traverses the wrap chain. If an intermediate caller wrapped the error with `fmt.Errorf`, `==` fails silently while `errors.Is` still finds the original sentinel error. Use `errors.Is` rather than `==` whenever the error may have passed through a wrapping function.

`errors.As` extracts a specific error type from the chain:

```
type PathError struct {
    Op   string
    Path string
}

func (e *PathError) Error() string {
    return fmt.Sprintf("%s: %s", e.Op, e.Path)
}
```

```go
err := someFunction()
var pathErr *PathError
if errors.As(err, &pathErr) {
    fmt.Printf("operation %s failed on path %s\n", pathErr.Op, pathErr.Path)
}
```

`errors.As` is used when you need to access fields on a custom error type buried in a wrap chain.

19 — Type Definitions and Aliases

Go creates named types from existing types with `type Name BaseType`. A type alias creates a synonym with `type Alias = BaseType`. The distinction is fundamental: a type definition creates a new type; an alias does not.

Type Definition

`type Name BaseType` creates a new distinct type based on an existing type:

```
type Celsius float64
type Fahrenheit float64
```

`Celsius` and `Fahrenheit` are distinct types. They cannot be assigned to each other without explicit conversion:

```
var c Celsius = 20
var f Fahrenheit = c    // compile error: cannot use Celsius as Fahrenheit
f = Fahrenheit(c)       // OK, explicit conversion
```

Methods on Named Types

A type definition enables adding methods to a primitive type:

```
type Duration int64

func (d Duration) String() string {
    return fmt.Sprintf("%dms", d)
}

func (d Duration) Seconds() float64 {
    return float64(d) / 1000
}

d := Duration(500)
fmt.Println(d.String())    // 500ms
fmt.Println(d.Seconds())   // 0.5
```

A new type has no methods inherited from the base type. Methods must be defined on the new type explicitly.

Type Alias

`type Alias = BaseType` creates a synonym — completely interchangeable with the base type in all contexts. An alias is not a new type; it is the same type under a different name. You can pass a

base type where an alias is expected, pass an alias where a base type is expected, assign between them freely, and use them as map keys or interface values without any conversion:

```
type StringList = []string

func printList(list StringList) {
    for _, s := range list {
        fmt.Println(s)
    }
}

printList([]string{"a", "b"})  // OK - passing base type where alias expected

var a StringList = []string{"x", "y"}
var b []string = a             // OK - assigning alias to base type

var m map[StringList]int = make(map[StringList]int)
m[b] = 1                       // OK - alias and base type used interchangeably as map key
```

Type aliases are used for renaming without creating a new type — useful for migration and clarity.

Comparison

	Type Definition	Type Alias
Creates new type	Yes	No
Assignable to base type	No (requires conversion)	Yes
Can define methods	Yes	No (methods go on base type)
Satisfies interfaces	Only if methods defined	Same as base type

Known Aliases

`byte` and `rune` (introduced in document 05) are aliases, not type definitions:

```
byte  // alias for uint8
rune  // alias for int32
```

They are interchangeable with their base types. `var b byte` and `var b uint8` declare the exact same type. You can assign between them, pass one where the other is expected, and use them in the same expressions — there is no conversion needed.

20 — Testing

Go's built-in testing framework requires no external dependencies. Test files end in `_test.go` and are excluded from normal builds.

Test Files and Functions

```
// calculator_test.go
package calculator

import "testing"

func TestAdd(t *testing.T) {
    result := Add(2, 3)
    if result != 5 {
        t.Errorf("Add(2, 3) = %d; want 5", result)
    }
}
```

Test functions must start with `Test` and accept `*testing.T`. `t.Errorf` marks the test as failed and continues execution. `t.Fatalf` marks it as failed and stops immediately.

Running Tests

```
go test ./...           # all tests in the module
go test ./pkg/          # tests in a specific package
go test -run TestAdd    # specific test (supports regex)
go test -cover          # with coverage report
```

Table-Driven Tests

The dominant Go testing pattern — a slice of structs defines input and expected output, iterated with range:

```
func TestMultiply(t *testing.T) {
    tests := []struct {
        name     string
        a, b     int
        expected int
    }{
        {"positive", 3, 4, 12},
        {"zero", 0, 5, 0},
```

```
        {"negative", -2, 3, -6},
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            result := Multiply(tt.a, tt.b)
            if result != tt.expected {
                t.Errorf("Multiply(%d, %d) = %d; want %d", tt.a, tt.b, result, tt.expected)
            }
        })
    }
}
```

`t.Run` creates a named sub-test for each table row. Run a specific sub-test with `go test -run TestMultiply/positive`.

Benchmarks

```
func BenchmarkAdd(b *testing.B) {
    for i := 0; i < b.N; i++ {
        Add(2, 3)
    }
}
```

`b.N` is set by the testing framework to produce a reliable timing. Run with `go test -bench=BenchmarkAdd`. The framework adjusts `b.N` across iterations until the measurement stabilizes.

Fuzz Testing

Built-in since Go 1.18:

```
func FuzzReverse(f *testing.F) {
    f.Fuzz(func(t *testing.T, input string) {
        reversed := Reverse(input)
        reversedAgain := Reverse(reversed)
        if reversedAgain != input {
            t.Fatalf("Reverse(Reverse(%q)) = %q", input, reversedAgain)
        }
    })
}
```

Run with `go test -fuzz=FuzzReverse`. The framework generates random inputs and mutates them to find failures. Seed the fuzzer with known inputs using `f.Add("example")`.

The fuzz function parameters can be any of these types: `bool`, `int`, `int8`, `int16`, `int32`, `int64`, `uint`, `uint8`, `uint16`, `uint32`, `uint64`, `uintptr`, `float32`, `float64`, `string`, `[]byte`. You can use multiple parameters in a single fuzz function:

```
func FuzzParse(f *testing.F) {
    f.Fuzz(func(t *testing.T, s string, n int, flag bool) {
        // fuzzer generates random values for all three parameters
    })
}
```

t.Helper

Mark a helper function with `t.Helper()` so that failure messages report the caller's location, not the helper's:

```
func assertEqual(t *testing.T, got, want int) {
    t.Helper()
    if got != want {
        t.Errorf("got %d, want %d", got, want)
    }
}

func TestAdd(t *testing.T) {
    assertEqual(t, Add(2, 3), 5)  // failure points here, not into assertEqual
}
```

Without `t.Helper()`, a failing assertion reports the line inside the helper function, making it harder to trace which test case triggered the failure.

Mocking

Go's interface-based design makes mocking straightforward — define an interface, implement a mock that satisfies it, and pass the mock to the code under test:

```
// Interface
type Store interface {
    Get(key string) (string, error)
}

// Mock implementation
type mockStore struct {
```

```
    data map[string]string
}

func (m mockStore) Get(key string) (string, error) {
    val, ok := m.data[key]
    if !ok {
        return nil, errors.New("not found")
    }
    return val, nil
}

// Usage in test
func TestHandler(t *testing.T) {
    store := mockStore{data: map[string]string{"key": "value"}}
    handler := NewHandler(store)
    // test with controlled data
}
```

No external mocking framework is needed. The interface acts as a contract between production code and test doubles. For larger projects, code generation tools like `mockgen` (from `go.uber.org/mock`) generate mock implementations from interface definitions automatically.

Integration Tests

Separate integration tests from unit tests using build tags. Add `//go:build integration` at the top of a test file to exclude it from `go test` by default:

```
//go:build integration

package db_test

import "testing"

func TestConnectToDatabase(t *testing.T) {
    // runs against real database
}
```

Run with `go test -tags=integration ./...`. This keeps slow or environment-dependent tests out of the normal test cycle while keeping them in the same repository.

21 — Goroutines

A goroutine is Go's lightweight concurrency primitive. The `go` keyword launches a function to run concurrently with the caller.

Launching a Goroutine

```
func greet(name string) {
    fmt.Printf("Hello, %s\n", name)
}

go greet("Alice")
greet("Bob")
```

The `go` statement returns immediately. The function runs concurrently in the background.

Main Goroutine Exit

The main goroutine exiting terminates the program immediately, regardless of running goroutines:

```
func main() {
    go func() {
        fmt.Println("this may not print")
    }()
    // program may exit before the goroutine runs
}
```

Synchronize with channels (document 22) or `sync.WaitGroup` (document 25) to wait for goroutines to complete.

Goroutines Are Cheap

Creating thousands of goroutines is normal. They are multiplexed onto OS threads by the Go runtime — not one-to-one with threads. A single OS thread can run many goroutines, and the runtime schedules them based on blocking and readiness.

Data Races

A data race occurs when multiple goroutines access the same memory without coordination, and at least one access is a write:

```
var count int

go func() {
```

```
    count++  // race: concurrent write
}()

go func() {
    count++  // race: concurrent write
}()

fmt.Println(count)  // race: concurrent read
```

Detect data races at runtime with the race detector:

```
go test -race
go run -race main.go
```

The race detector instruments memory accesses and reports conflicts with stack traces. Use it during development — it has a performance cost but catches real bugs.

Goroutine Leaks

A goroutine that blocks forever accumulates silently and consumes memory. This happens when a goroutine waits on a channel that is never closed or sent to:

```
func worker(done chan struct{}) {
    <-done  // blocks forever if done is never closed
}

go worker(nil)  // leaked - nil channel blocks forever
```

The fix is to let the goroutine listen for cancellation alongside its blocking operation using `select` and `ctx.Done()`. See document 23 for the pattern.

Waiting for Goroutines

The `go` statement returns immediately. To wait for goroutines to finish, use one of these patterns:

`sync.WaitGroup` — wait for a known number of goroutines (document 25):

```
var wg sync.WaitGroup

for _, url := range urls {
    wg.Add(1)
    go func(u string) {
        defer wg.Done()
        fetch(u)
```

```
    }(url)
}

wg.Wait()  // blocks until all goroutines call Done()
```

Channel signaling — wait for a single result (document 22):

```
ch := make(chan string)
go func() {
    ch <- fetch("https://example.com")
}()

result := <-ch  // blocks until the goroutine sends
```

Context cancellation — wait with a timeout (document 23):

```
ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()

go worker(ctx)
// worker exits when it finishes or the context times out
```

22 — Channels

Accessing shared variables from multiple goroutines causes race conditions. Channels solve this by passing values between goroutines instead of sharing memory. They are the synchronization mechanism in Go's concurrency model.

Declaration and Creation

`chan T` is a channel that carries values of type `T`. The zero value of a channel is `nil`:

```
var ch chan int  // nil channel
```

A nil channel blocks on all operations — sends and receives hang forever. It is effectively a channel that can never complete an operation:

```
var ch chan int  // nil

go func() {
    ch <- 42  // blocks forever
}()

<-ch  // blocks forever
```

`make()` creates a usable channel. It takes a type and an optional capacity:

```
ch := make(chan int)       // unbuffered channel
ch := make(chan int, 10)   // buffered channel, capacity 10
```

Send and Receive

Both send and receive use the `<-` operator. The difference is the position of the channel name: on the left for send, on the right for receive.

Send puts a value into a channel. The value can be a variable, literal, or expression:

```
ch <- 42         // channel on the left - send
ch <- x + y
ch <- compute()
```

Receive reads a value from a channel and assigns it to a variable:

```
value := <-ch    // channel on the right - receive
```

Both operations block until the other side is ready. A send waits until a receiver is available to consume the value. A receive waits until a value is available to read.

Directional Channels

`chan<- T` (send-only) and `<-chan T` (receive-only) are proper channel types that restrict a channel to either sending or receiving. They are compatible with the bidirectional type `chan T`, so a normal channel can be assigned to a send-only or receive-only variable. They can be used as variable types, struct fields, and anywhere a type is expected:

```
var sendCh chan<- int   // send-only variable
var recvCh <-chan int   // receive-only variable

sendCh <- 42            // OK
<-sendCh                // compile error

<-recvCh                // OK
recvCh <- 42            // compile error
```

A bidirectional channel (`chan T`) converts to either directional type. A directional channel cannot convert back:

```
ch := make(chan int)
var sendCh chan<- int = ch   // OK
var recvCh <-chan int = ch   // OK
```

They are most commonly used in function signatures to restrict what a caller can do with a channel:

```
func producer(sendCh chan<- int) {  // send-only
    sendCh <- 42
}

func producer() <-chan int {        // returns receive-only
    ch := make(chan int)
    go func() { ch <- 42 }()
    return ch
}
```

Unbuffered Channels

An unbuffered channel blocks both send and receive until the other side is ready — this is the synchronization mechanism:

```
ch := make(chan int)

go func() {
    fmt.Println("sending")
```

```
    ch <- 42
    fmt.Println("sent")
}()

fmt.Println("waiting")
value := <-ch
fmt.Println("received", value)
```

The send blocks until the receive is ready. The receive blocks until the send is ready. They synchronize at the channel operation.

Buffered Channels

A buffered channel sends block only when the buffer is full and receives block only when empty:

```
ch := make(chan int, 2)

ch <- 1   // OK, buffer has space
ch <- 2   // OK, buffer full
ch <- 3   // blocks, buffer is full

<-ch      // unblocks the send
```

Close

`close(ch)` signals that no more values will be sent. A receive from a channel returns two values: the received value and a boolean `ok` that is `true` if the channel is open and `false` after the last buffered item has been consumed from a closed channel. This lets the receiver detect when the sender is done:

```
ch := make(chan int)

go func() {
    ch <- 1
    ch <- 2
    ch <- 3
    close(ch)
}()

for {
    value, ok := <-ch
    if !ok {
```

```
        break  // channel closed, no more values
    }
    fmt.Println(value)  // 1, 2, 3
}
```

Sending on a closed channel panics. Receiving from a closed channel does not — it returns immediately, either with a buffered value or with the zero value and `ok = false`.

Range over Channels

`for..range` on a channel receives values until the channel is closed. It is the idiomatic shorthand for the close-and-drain pattern shown above:

```
for value := range ch {
    fmt.Println(value)
}
// loop exits when the channel is closed and drained
```

The loop blocks on each iteration waiting for the next value. It only terminates when the sender calls `close(ch)` and all remaining values have been received. A channel that is never closed will cause the loop to block forever.

You can discard the value with the blank identifier if you only need to wait for the channel to close:

```
for range doneCh {
    // process signals
}
// exits when doneCh is closed
```

Select

`select` waits on multiple channel send and receive operations simultaneously:

```
select {
case msg := <-ch1:  // receive
    fmt.Println("from ch1:", msg)
case msg := <-ch2:  // receive
    fmt.Println("from ch2:", msg)
case ch3 <- 42:     // send
    fmt.Println("sent to ch3")
}
```

If multiple cases are ready, `select` chooses one at random. If none are ready, it blocks.

Select with Default

A `default` case makes `select` non-blocking:

```
select {
case msg := <-ch:
    fmt.Println("received:", msg)
default:
    fmt.Println("no message available")
}
```

Select with Timeout

```
select {
case msg := <-ch:
    fmt.Println("received:", msg)
case <-time.After(5 * time.Second):
    fmt.Println("timed out")
}
```

`time.After` returns a channel that receives the current time after the duration elapses.

23 — Context

When a function calls other functions that perform I/O or block, the caller needs a way to cancel the entire operation. `context.Context` is that mechanism — it carries cancellation intent and deadlines down through the call stack.

How It Works

A context is a **shared state object**, not an enforcement mechanism. When a timeout fires or a cancel function is called, the context marks itself as cancelled. It does not stop any running goroutine or abort any operation. Each function in the call chain must actively check the context and choose to stop.

The two methods you'll use are **`ctx.Done()`** (detects that cancellation happened) and **`ctx.Err()`** (returns why — `context.Canceled` or `context.DeadlineExceeded`). See the interface definition below for the full picture.

If a function in the chain ignores the context, cancellation has no effect at that layer. The mechanism only works because every layer checks and propagates the cancellation.

The Context Interface

By convention, `context.Context` is passed as the first argument to any function that may block or do I/O, and is named `ctx`:

```
func fetch(ctx context.Context, url string) (string, error) {
    // implementation
}
```

The actual interface has four methods:

```
type Context interface {
    Deadline() (deadline time.Time, ok bool)
    Done()     chan struct{}
    Err()      error
    Value(key any) any
}
```

- **`Deadline()`** returns the time at which the context will be automatically cancelled, and a boolean indicating whether a deadline is set. Libraries use this to compute how long they have before the context expires (e.g. setting a socket timeout).
- **`Done()`** returns `chan struct{}` — a channel that closes when the context is cancelled. You listen on it with `select` to detect **that** cancellation happened. A channel is used instead of a boolean so cancellation can be multiplexed with other async operations in a single `select` — waiting on I/O, a timer, and cancellation all at once, without busy-polling. A closed channel

is always ready to receive, so `case <-ctx.Done()` becomes eligible the moment the context is cancelled.

- **`Err()`** returns `error` — `nil` while active, `context.Canceled` if manually cancelled, or `context.DeadlineExceeded` if a timeout or deadline fired. These are sentinel errors compared with `errors.Is()`. Call this **after** `ctx.Done()` fires to find out **why** the context was cancelled.
- **`Value()`** retrieves a value stored in the context by key. Covered in the Context Value section below.

Creating Contexts

Contexts form a tree: each context has a parent, and cancellation propagates downward. When a function receives a context and needs to add a deadline or timeout, it derives a new context from the one it received — so it never knows (or needs to know) whether an earlier layer already set a deadline. The child is cancelled by whichever happens first: its own trigger or its parent's cancellation.

`context.Background()` is the **root** of this tree — an empty context that never cancels on its own, has no deadline, and carries no values. It is the base case when no context exists yet (typically at the entry point of a request or program).

Several builder functions create derived contexts. All of them take an existing context as their first argument and return a new context plus a cancel function:

- `WithCancel(parent)` — manual cancellation via the returned `cancel()` function
- `WithTimeout(parent, duration)` — auto-cancels after a duration
- `WithDeadline(parent, time)` — auto-cancels at a specific time
- `WithValue(parent, key, value)` — attaches a key-value pair (discouraged)

```go
func handleRequest() {
    // Entry point: derive from Background() - the root.
    ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
    defer cancel()
    doWork(ctx)
}

func doWork(parentCtx context.Context) {
    // Inner layer: derive from the received context.
    // This child is cancelled if parentCtx's 5s timeout fires first,
    // or after 2s - whichever comes first.
    ctx, cancel := context.WithTimeout(parentCtx, 2*time.Second)
    defer cancel()
    // ... do work with ctx ...
}
```

Always call `defer cancel()` immediately after creating a derived context. `cancel()` is a cleanup operation: it releases the context's internal resources (timers, tree nodes) so they can be garbage collected. For `WithTimeout` and `WithDeadline`, this matters when the operation finishes before the deadline — `cancel()` stops the timer early. It is safe to call on an already-cancelled context. Forgetting to call it leaks resources.

Calling `cancel()` on **any** derived context — including `WithTimeout` and `WithDeadline` — triggers immediate manual cancellation, regardless of whether the timer has fired. It closes `ctx.Done()`, waking up all children listening on it. `ctx.Err()` returns `context.Canceled` (not `DeadlineExceeded`), because the cancellation came from `cancel()`, not the timer.

WithCancel

Returns a context that can be cancelled manually by calling the returned `cancel()` function:

```
ctx, cancel := context.WithCancel(context.Background())
defer cancel()

go func() {
    result, err := fetch(ctx, url)
    // fetch checks ctx.Done() and stops if cancelled
}()

// later, when the result is no longer needed:
cancel()
```

WithTimeout

Returns a context that auto-cancels after a duration elapses:

```
ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()

result, err := fetch(ctx, url)
if err != nil {
    if errors.Is(err, context.DeadlineExceeded) {
        fmt.Println("request timed out")
    }
}
```

WithDeadline

Returns a context that auto-cancels at a specific absolute time:

```
deadline := time.Now().Add(10 * time.Second)
ctx, cancel := context.WithDeadline(context.Background(), deadline)
defer cancel()
```

Respecting Context

1. Pass the context to a function that already checks it. Many standard library functions watch the context internally — for example, `http.Client.Do()` aborts the request if the context is cancelled. You just pass the context and the library handles the rest.

2. Check `ctx.Err()` before doing work. The simplest, most deterministic check — no channels involved:

```
func process(ctx context.Context) error {
    if ctx.Err() != nil {
        return ctx.Err()
    }
    // do work
    return nil
}
```

3. Multiplex cancellation with other channels using `select`. This is the main reason `ctx.Done()` returns a channel — it lets you wait on cancellation alongside other async operations in a single `select`:

```
func worker(ctx context.Context, jobs <-chan Job) error {
    for {
        select {
        case <-ctx.Done():
            return ctx.Err()
        case job, ok := <-jobs:
            if !ok {
                return nil // channel closed, no more work
            }
            // process job
        }
    }
}
```

`select` evaluates all cases and picks one that is ready. `<-ctx.Done()` becomes ready the moment the context is cancelled, whether by timeout, deadline, or an explicit `cancel()` call. Without the channel, this kind of multiplexing would require polling `ctx.Err()` in a loop.

Complete Example

A master-worker pattern: the master creates a context with a timeout, a dispatch function derives a tighter deadline from it, and a worker checks the context before processing each job.

```
package main

import (
    "context"
    "fmt"
    "time"
)

// worker processes jobs from a channel, stopping if the context is cancelled.
func worker(ctx context.Context, jobs <-chan int) (int, error) {
    count := 0
    for {
        select {
        case <-ctx.Done():
            return count, ctx.Err()
        case <-jobs:
            count++
        }
    }
}

// dispatch sends jobs to the worker indefinitely, until the context fires.
func dispatch(ctx context.Context, jobs chan<- int) (int, error) {
    // Derive a context with a longer timeout from the received one.
    // The parent expires at 250ms, so this 500ms deadline never fires.
    ctx, cancel := context.WithTimeout(ctx, 500*time.Millisecond)
    defer cancel()

    count := 0
    for {
        select {
        case <-ctx.Done():
            return count, ctx.Err()
        case jobs <- count:
            count++
        }
```

```
    }
}

func main() {
    // Entry point: create context from Background().
    ctx, cancel := context.WithTimeout(context.Background(), 250*time.Millisecond)
    defer cancel()

    jobs := make(chan int)
    go worker(ctx, jobs)

    sent, sendErr := dispatch(ctx, jobs)
    fmt.Printf("dispatch: sent %d jobs - %v\n", sent, sendErr)
}
```

Output (exact job count varies by machine):

```
dispatch: sent 1259919 jobs - context deadline exceeded
```

What happened:

1. `main` creates a context with a 250ms timeout from `Background()`.
2. `dispatch` derives a 500ms deadline from the received context — but the parent's 250ms is tighter, so that's the one that wins.
3. After 250ms, the parent timer fires and closes `ctx.Done()` automatically. Both the worker and dispatch see it at the same time — the worker because it uses the parent directly, dispatch because cancellation propagates down the tree to its derived context.
4. Both `select` statements pick `ctx.Done()` and return. The last send never completed, so both counts match.
5. The infinite loops were stopped solely by the context — no counters, limits, or flags needed.

Context Value

`context.WithValue` exists for passing request-scoped data. It is not covered here by design — it is rarely appropriate and encourages misuse. Prefer explicit parameters.

24 — Cleanup Patterns

Go's `defer` combines with error handling and context for resource cleanup. The patterns are consistent and appear throughout real code.

Open, Check, Defer Close

The canonical resource pattern — open, check error, defer close, in that exact order:

```
func processFile(path string) error {
    f, err := os.Open(path)
    if err != nil {
        return err
    }
    defer f.Close()

    data, err := io.ReadAll(f)
    if err != nil {
        return err
    }

    // use data
    return nil
}
```

The `defer f.Close()` runs when the function returns, regardless of which return statement is taken. The error check before the defer ensures the resource was opened successfully before scheduling cleanup.

Context Cancellation

`defer cancel()` placed immediately after `context.WithCancel` or `context.WithTimeout`:

```
func handleRequest(w http.ResponseWriter, r *http.Request) {
    ctx, cancel := context.WithTimeout(r.Context(), 30*time.Second)
    defer cancel()

    result, err := doWork(ctx)
    if err != nil {
        http.Error(w, err.Error(), 500)
        return
    }
```

```
    fmt.Fprintf(w, "%s", result)
}
```

Multiple Resources

When opening multiple resources, defer each one after its error check:

```
func copyFiles(src, dst string) error {
    in, err := os.Open(src)
    if err != nil {
        return err
    }
    defer in.Close()

    out, err := os.Create(dst)
    if err != nil {
        return err
    }
    defer out.Close()

    _, err = io.Copy(out, in)
    return err
}
```

If `os.Create` fails, `in.Close()` runs via defer before the function returns. If both succeed, both close functions run in LIFO order when the function returns.

Early Return

Early return on error is idiomatic — flat code without nesting is preferred:

```
func handle(r *http.Request) error {
    ctx, cancel := context.WithTimeout(r.Context(), 10*time.Second)
    defer cancel()

    data, err := fetch(ctx, r.URL.String())
    if err != nil {
        return err
    }

    parsed, err := parse(data)
    if err != nil {
```

```
        return err
    }

    result, err := compute(parsed)
    if err != nil {
        return err
    }

    return store(result)
}
```

This pattern avoids deeply nested error checks and keeps the happy path linear. Each error check returns immediately, and defer handles all cleanup.

25 — Sync Package

The `sync` package provides non-channel concurrency primitives for mutual exclusion and coordination. Channels communicate between goroutines; `sync` coordinates shared-memory access.

Mutex

`sync.Mutex` provides mutual exclusion — only one goroutine holds the lock at a time:

```
var (
    mu    sync.Mutex
    count int
)

func increment() {
    mu.Lock()
    defer mu.Unlock()
    count++
}

func getCount() int {
    mu.Lock()
    defer mu.Unlock()
    return count
}
```

`defer mu.Unlock()` ensures the lock is released even if the function panics. A Mutex is not reentrant — calling `Lock()` twice from the same goroutine deadlocks.

RWMutex

`sync.RWMutex` allows concurrent readers or an exclusive writer:

```
var mu sync.RWMutex
var config map[string]string

func getConfig(key string) string {
    mu.RLock()
    defer mu.RUnlock()
    return config[key]
}

func setConfig(key, value string) {
```

```
    mu.Lock()
    defer mu.Unlock()
    config[key] = value
}
```

Multiple readers can hold `RLock` simultaneously. A writer's `Lock` excludes all readers and other writers. Use `RWMutex` when reads significantly outnumber writes.

WaitGroup

`sync.WaitGroup` waits for a collection of goroutines to finish:

```
var wg sync.WaitGroup

urls := []string{"https://a.com", "https://b.com", "https://c.com"}

for _, url := range urls {
    wg.Add(1)
    go func(u string) {
        defer wg.Done()
        fetch(u)
    }(url)
}

wg.Wait()  // blocks until all goroutines call Done()
fmt.Println("all done")
```

`Add(n)` sets the counter. `Done()` decrements by 1. `Wait()` blocks until the counter reaches zero. Pass `WaitGroup` by pointer — copying it resets the counter.

Once

`sync.Once` executes a function exactly once across goroutines:

```
var (
    once     sync.Once
    instance *Database
    initErr  error
)

func getDB() (*Database, error) {
    once.Do(func() {
        instance, initErr = connectDB()
```

```
    })
    return instance, initErr
}
```

The first call to `Do` runs the function. Subsequent calls return immediately without running it again. `Do` is safe to call from multiple goroutines simultaneously — if a second goroutine calls `Do` while the first is still executing the function, it blocks until the function completes. This guarantees that `instance` and `initErr` are assigned before any caller reaches the `return` statement.

Sync.Map

`sync.Map` is a specialized concurrent map optimized for two scenarios:

1. **Keys are written once, then read many times** — the map "promotes" frequently-read entries to a lock-free read path.
2. **Different goroutines access disjoint key sets** — writes to different keys don't block each other.

```
var cache sync.Map

cache.Store("key", "value")
value, ok := cache.Load("key")
cache.Delete("key")

cache.Range(func(key, value any) bool {
    fmt.Printf("%v: %v\n", key, value)
    return true  // continue iteration; return false to stop
})
```

Trade-off vs. map + `sync.Mutex`: A regular map with a mutex uses a single global lock — all reads and writes contend on it. `sync.Map` uses internal per-entry locking, so reads of "hot" entries are lock-free and writes to different keys don't block each other. The cost is higher memory overhead, slower writes under heavy contention, and a more verbose API (no `m[key]` syntax). For most use cases, a map with a `sync.Mutex` or `sync.RWMutex` is simpler and faster.

Atomic Operations

`sync/atomic` provides lock-free primitives for safe concurrent access to individual variables. Operations are guaranteed to complete without interruption.

Type-based API (Go 1.19+, preferred):

```
var (
    count atomic.Int64
```

```
    flag  atomic.Bool
)

count.Add(1)
count.Load()
count.Store(0)

flag.Store(true)
flag.Load()
flag.Swap(false)
```

Available types: `atomic.Int32`, `atomic.Int64`, `atomic.Int64`, `atomic.Uint32`, `atomic.Uint64`, `atomic.Uintptr`, `atomic.Bool`, `atomic.String`, `atomic.Pointer[T]`.

Compare-and-swap atomically updates a value only if it currently holds an expected value:

```
// Returns true if the swap happened, false if the value was already different
ok := count.CompareAndSwap(5, 10)
```

Use atomics for simple counters, flags, and single-variable coordination. For operations that involve multiple variables or complex invariants, use a mutex — atomics cannot guarantee consistency across more than one variable.

Pool

`sync.Pool` reuses temporary objects to reduce allocation pressure and GC load. You store objects when done and retrieve them when needed, avoiding repeated allocations:

```
var bufPool = sync.Pool{
    New: func() any { return new(bytes.Buffer) },
}

buf := bufPool.Get().(*bytes.Buffer)
defer bufPool.Put(buf)
// use buf
```

`Get()` returns a previously stored object or calls `New()` if the pool is empty. `Put()` returns an object to the pool for reuse. Objects in the pool can be discarded by the garbage collector at any time — the pool is not a guaranteed cache. Reset all pooled objects with `pool.Purge()` (e.g., before a configuration change).

Typical use: buffers, parsers, or other expensive-to-allocate objects created and discarded in hot paths.

26 — Init Function

`func init()` runs automatically before `main()`. It has no parameters and no return values. Each package may have multiple `init` functions across its files; all run at startup.

Basic Usage

```
var colors = map[string]int{
    "red": 0,
    "green": 1,
    "blue": 2,
}

func init() {
    colors["yellow"] = 3
    colors["purple"] = 4
}
```

`init` runs after package-level variable initialization and before `main()`. It is used to set up state that cannot be expressed as a simple variable declaration.

Execution Order

Within a single file, `init` functions run top-to-bottom in the order they appear.

Across files in the same package, files are processed in lexical (alphabetical) order by filename. For a package with `a.go` containing two `init` functions and `b.go` containing one, the order is: `a.go` init #1, `a.go` init #2, `b.go` init.

Across packages, dependencies run first, then the importing package:

`package A imports package B`

Execution order: B's variables, B's `init`, A's variables, A's `init`, then `main()`.

Relying on init order is fragile. If correctness depends on a specific sequence, use explicit initialization instead.

Typical Uses

Registering drivers or plugins:

```
func init() {
    RegisterDriver("mysql", &MySQLDriver{})
}
```

Initializing package-level state:

```
var config Config

func init() {
    var err error
    config, err = loadDefaultConfig()
    if err != nil {
        log.Fatalf("failed to load config: %v", err)
    }
}
```

Validating invariants at startup:

```
func init() {
    if minBufferSize > maxBufferSize {
        log.Fatalf("invalid buffer size configuration")
    }
}
```

Limitations

`init` cannot be called explicitly from user code. It is the only function name allowed to appear multiple times — a single file may contain several `init` functions, and multiple files in the same package may each define their own. The compiler treats `init` as a special case, collecting all definitions and generating a hidden call sequence. Normal functions must have unique names within a package.

Circular imports prevent `init` from running at all — if package A imports package B and package B imports package A, the compiler rejects the code.

`init` functions make dependencies implicit and harder to trace. Use them sparingly — explicit initialization in `main()` or constructor functions is more testable and easier to reason about.

27 — Reflection

The `reflect` package provides runtime type inspection. It is used to examine struct fields and their tags, determine types at runtime, and manipulate values dynamically.

TypeOf and ValueOf

```
import "reflect"

name := "Alice"
t := reflect.TypeOf(name)    // reflect.Type
v := reflect.ValueOf(name)   // reflect.Value

fmt.Println(t.Kind())  // string
fmt.Println(v.String()) // Alice
```

`TypeOf` returns the dynamic type. `ValueOf` returns a wrapper that can be inspected and manipulated.

Kind

`value.Kind()` returns the underlying kind: `Struct`, `Slice`, `Map`, `Ptr`, `Int`, `String`, etc. Kind is the base category, not the specific type — both `int` and `int64` have kind `Int`:

```
type Duration int64
d := Duration(500)
v := reflect.ValueOf(d)
fmt.Println(v.Kind())  // Int (not Duration)
```

Inspecting Structs

The most common use of reflection — inspecting struct fields and their tags:

```
type User struct {
    Name  string `json:"name"`
    Age   int    `json:"age,omitempty"`
    Email string `json:"email"`
}

u := User{Name: "Alice", Age: 30, Email: "alice@example.com"}
t := reflect.TypeOf(u)
v := reflect.ValueOf(u)

for i := 0; i < t.NumField(); i++ {
```

```
    field := t.Field(i)
    value := v.Field(i)
    tag := field.Tag.Get("json")
    fmt.Printf("%s: %v (json: %s)\n", field.Name, value.Interface(), tag)
}
// Name: Alice (json: name)
// Age: 30 (json: age,omitempty)
// Email: alice@example.com (json: email)
```

This is how `encoding/json` and similar packages work — they read struct tags to control serialization.

Setting Values

Setting a value through reflection requires a pointer:

```
count := 5
v := reflect.ValueOf(&count).Elem()
v.SetInt(10)
fmt.Println(count)  // 10
```

`ValueOf(&count)` returns a `reflect.Value` holding a pointer. `Elem()` dereferences it to get the underlying value. `SetInt` modifies the actual variable.

Calling `SetInt` on a non-addressable value (one obtained from `ValueOf(x)` without taking the address) panics.

Available setters for primitive types:

Method	Applies to
`SetInt`	`int`, `int8`, `int16`, `int32`, `int64`
`SetUint`	`uint`, `uint8`, `uint16`, `uint32`, `uint64`, `uintptr`
`SetFloat`	`float32`, `float64`
`SetBool`	`bool`
`SetString`	`string`
`SetBytes`	`[]byte`
`SetComplex`	`complex64`, `complex128`

Each setter only works on the matching kind — calling `SetInt` on a `float64` value panics.

`Set(reflect.Value)` sets a value from another `reflect.Value`. It works for any type that is assignable: interfaces, structs, slices, maps, pointers, channels, and functions. The source value must be assignable to the destination type.

Safety

Reflection bypasses compile-time type checking. Errors surface at runtime as panics:

```
v := reflect.ValueOf("hello")
v.SetInt(42)  // panic: reflect: reflect.SetInt of string
```

Generics (document 29) have replaced many historical uses of reflection. Modern Go code uses reflection primarily for serialization frameworks and frameworks that need to inspect struct tags.

28 — Unsafe

The `unsafe` package bypasses Go's type system and memory safety guarantees. Its presence in application code is a red flag requiring scrutiny.

What Unsafe Provides

```
import "unsafe"
```

`unsafe.Pointer` converts between pointer types that would otherwise be incompatible:

```
var i int = 42
p := (*byte)(unsafe.Pointer(&i))
fmt.Println(*p)  // first byte of the int's memory representation
```

`unsafe.Sizeof(x)` returns the memory size of a value in bytes:

```
unsafe.Sizeof(int64(0))   // 8
unsafe.Sizeof(true)       // 1
```

`unsafe.Offsetof(s.Field)` returns a struct field's byte offset:

```
type Point struct {
    X int
    Y int
}
unsafe.Offsetof(Point{}.X)  // 0
unsafe.Offsetof(Point{}.Y)  // 8 (on 64-bit)
```

Consequences

Code using `unsafe` is not protected by Go's compatibility guarantees. It may break across Go versions, architectures, or compiler updates. The garbage collector does not track `unsafe.Pointer` values — a pointer obtained through `unsafe` does not keep its target alive.

Legitimate Uses

Legitimate uses are narrow:

- Interoperating with C via cgo
- Implementing low-level runtime primitives
- Certain performance-critical data structure operations

The standard library uses `unsafe` internally for these purposes. This is categorically different from application code using it — the standard library is written by the same team that controls the runtime and can verify safety manually.

Recognition

When reviewing code, `unsafe` in application logic warrants a careful review of what it is doing and why a safe alternative is not sufficient. Most uses of `unsafe` can be replaced with typed code, reflection, or a redesign.

29 — Generics

Type parameters allow writing functions and parametrized types that work with multiple types without losing type safety. Go added generics in version 1.18.

Generic Functions

Type parameters are listed in square brackets before the function parameters:

```
func First[T any](slice []T) T {
    if len(slice) == 0 {
        return *new(T)  // zero value of T
    }
    return slice[0]
}

First([]int{1, 2, 3})           // 1
First([]string{"a", "b", "c"})  // "a"
```

`any` is the unconstrained form — it accepts any type.

Constraints

Constraints restrict which types are allowed:

```
func Minimum[T int | float64](a, b T) T {
    if a < b {
        return a
    }
    return b
}

Minimum(3, 5)        // 3
Minimum(1.5, 2.5)    // 1.5
Minimum("a", "b")    // compile error: string not allowed
```

`comparable` allows types that support `==` and `!=`:

```
func Contains[T comparable](slice []T, target T) bool {
    for _, v := range slice {
        if v == target {
            return true
        }
    }
```

```
    return false
}
```

Interface Constraints

Any interface can serve as a constraint. The type parameter must satisfy the interface:

```
import "io"

func ReadAll[T io.Reader](r T) []byte {
    var buf bytes.Buffer
    io.Copy(&buf, r)
    return buf.Bytes()
}
```

Interface constraints and type unions can be combined:

```
func Process[T ~int | ~float64 | io.Reader](v T) {
    // v can be int, float64, or any type implementing io.Reader
}
```

The `~` prefix means "any underlying type." `~int` matches `int` and any named type with `int` as its underlying type (e.g., `type MyInt int`).

Built-in Constraints

Go provides three built-in constraints:

Constraint	Meaning
`any`	Any type
`comparable`	Types that support `==` and `!=` (all types except slices, maps, and functions)
`cmp.Ordered` (Go 1.21+)	Types that support ordering (`<`, `>`, etc.) — `int`, `float`, and `string` types

```
import "cmp"

func Minimum[T cmp.Ordered](a, b T) T {
    if a < b {
        return a
    }
    return b
}
```

You can also define generic types on structs, and their methods carry the type parameter — methods cannot introduce their own type parameters independent of the struct:

```
type Stack[T any] struct {
    items []T
}

func (s *Stack[T]) Push(item T) {
    s.items = append(s.items, item)
}

func (s *Stack[T]) Pop() (T, bool) {
    if len(s.items) == 0 {
        var zero T
        return zero, false
    }
    item := s.items[len(s.items)-1]
    s.items = s.items[:len(s.items)-1]
    return item, true
}

stack := &Stack[int]{}
stack.Push(1)
stack.Push(2)
stack.Pop()  // 2, true
```

Methods on generic types use the type's parameter. Generic methods — methods with their own type parameters independent of the type — are not supported in Go.

Standard Library Generics

The `slices` and `maps` packages (Go 1.21+) provide generic utility functions:

```
import "slices"

slices.Sort([]int{3, 1, 2})
slices.Contains([]string{"a", "b"}, "b")  // true
slices.ContainsFunc([]int{1, 2, 3}, func(n int) bool { return n > 2 })  // true

import "maps"

maps.Clone(map[string]int{"a": 1, "b": 2})
```

```
maps.Equal(map[string]int{"a": 1}, map[string]int{"a": 1})  // true
```

Most application code does not need to define generics — it uses them via the standard library. Defining custom generics is appropriate when the same logic applies to multiple types and type safety matters.

30 — Logging

Go provides two logging packages: `log` (simple, text-based) and `log/slog` (structured, Go 1.21+). Use `log` for scripts, CLI tools, and simple programs. Use `slog` for services, libraries, and any code where logs are parsed by tools. Both write to `os.Stderr` by default.

Log

The `log` package writes timestamped text lines to `os.Stderr`. It has no concept of log levels — just formatting variants and a fatal variant that exits:

```
log.Print("message")          // fmt.Sprint-style (no spaces between args)
log.Println("message")        // fmt.Sprintln-style (spaces between args)
log.Printf("count: %d", 42)   // fmt.Sprintf-style (format string)
```

All three add a trailing newline to the output line. The difference is how arguments are formatted, following the same pattern as `fmt.Print`/`Println`/`Printf`.

`log.Fatal` (and `Fatalf`, `Fatalln`) prints and calls `os.Exit(1)`. It runs deferred functions before exiting, so deferred cleanup still executes:

```
log.Fatal("cannot start server")      // prints and exits
log.Fatalln("cannot start server")    // prints with newline and exits
log.Fatalf("failed to open %s: %v", path, err)  // formatted, exits
```

The default logger writes to `os.Stderr` with a timestamp and newline. You can customize output with `log.SetPrefix`, `log.SetFlags`, or create a custom logger with `log.New`:

```
logger := log.New(os.Stdout, "DEBUG: ", log.Ldate|log.Ltime)
logger.Println("custom output")
```

`SetPrefix` and `SetFlags` modify the default logger globally:

```
log.SetPrefix("myapp: ")
log.SetFlags(log.Ldate | log.Ltime | log.Lshortfile)
log.Println("error occurred")
// Output: myapp: 2024-01-15 10:30:00 main.go:42: error occurred
```

The `Ldate`, `Ltime`, `Lmicroseconds`, `Llongfile`, and `Lshortfile` flags control what appears before the message. Combine with `|`. The default is `Ldate | Ltime | Lmicroseconds`.

Slog

`log/slog` (Go 1.21+) produces structured logs — each record carries a level, message, and key-value attributes. It is the idiomatic logging approach for new code:

```
slog.Info("server started", "addr", ":8080", "version", "1.2.3")
slog.Warn("slow query", "duration", 1200*time.Millisecond)
slog.Error("connection failed", "err", err)
```

Output is structured text by default (parseable by tools):

```
time=2024-01-15T10:30:00Z level=INFO msg="server started" addr=":8080" version="1.2.3"
```

Handler

The handler controls output format. There are 2 handlers in the standard libriary:

- `slog.NewTextHandler` writes structured text;
- `slog.NewJSONHandler` writes JSON.

Both take an `io.Writer` and optional `*slog.HandlerOptions` (pass `nil` for defaults):

```
textHandler := slog.NewTextHandler(os.Stdout, nil)
textLogger := slog.New(textHandler)
textLogger.Info("text handler output")
// time=2024-01-15T10:30:00Z level=INFO msg="text handler output"

jsonHandler := slog.NewJSONHandler(os.Stdout, nil)
jsonLogger := slog.New(jsonHandler)
jsonLogger.Info("structured JSON output")
// {"time":"2024-01-15T10:30:00Z","level":"INFO","msg":"structured JSON output"}
```

Make a logger the process-wide default with `slog.SetDefault(logger)`. After calling it, the package-level functions `slog.Info()`, `slog.Warn()`, `slog.Error()` use your configured logger instead of the built-in default.

HandlerOptions

`HandlerOptions` configures a handler. It has three fields:

- **`Level`** — minimum log level. Four levels exist, from lowest to highest severity: `DEBUG` (-4), `INFO` (0), `WARN` (4), `ERROR` (8). The default minimum is `INFO` — `DEBUG` messages are discarded.
- **`AddSource`** — when `true`, adds file and line number to each log entry.
- **`ReplaceAttr`** — function to transform attributes before output (advanced, rarely used).

```
opts := &slog.HandlerOptions{
    Level:     slog.LevelDebug,
    AddSource: true,
}
debugHandler := slog.NewTextHandler(os.Stdout, opts)
```

```
debugLogger := slog.New(debugHandler)
debugLogger.Debug("only visible with LevelDebug")
debugLogger.Info("always visible")
```

The idiomatic form passes the options inline: `slog.NewTextHandler(os.Stdout, &slog.HandlerOptions{Level: slog.LevelDebug})`.

With

`With` attaches attributes to every subsequent log from a logger, avoiding repetition:

```
logger := slog.With("requestID", req.ID)
logger.Info("received request")
logger.Info("processing")
logger.Info("response sent")
```

Each call inherits the `requestID` attribute. `With` returns a new logger — it does not modify the original.

31 — Common Gotchas

Corner cases and subtle behaviors that trip up experienced developers. Each entry describes the behavior, why it happens, and a minimal example.

Time.Duration Is Just Int64

`time.Duration` is `int64` nanoseconds. Arithmetic works, but the type prevents mixing with plain integers:

```
d := 5 * time.Second
n := int64(d)  // 5000000000, explicit conversion needed
```

Strings.Builder and Bytes.Buffer Are Not Concurrency-Safe

Do not share `strings.Builder` or `bytes.Buffer` across goroutines:

```
var buf strings.Builder
go func() { buf.WriteString("a") }()  // data race
go func() { buf.WriteString("b") }()  // data race
```

JSON Unmarshaling Numbers as Float64

When unmarshaling JSON into an `any` (or `map[string]any`), the `encoding/json` package has no type information to guide it. It decodes **all** JSON numbers as `float64`, regardless of whether they look like integers:

```
var data any
json.Unmarshal([]byte(`{"count": 10000000000}`), &data)
m := data.(map[string]any)
fmt.Printf("%T\n", m["count"])  // float64, not int
```

`float64` cannot precisely represent integers larger than 2^{53}. Values like IDs, timestamps, or large counters silently lose precision:

```
m["count"].(float64)  // 10000000000 may not equal the original
```

Two fixes:

Use a typed struct so the decoder knows the target type:

```
type Payload struct {
    Count int64 `json:"count"`
}
var p Payload
```

```
json.Unmarshal([]byte(`{"count": 10000000000}`), &p)
// p.Count is int64, exact value preserved
```

Or use `json.Decoder.UseNumber()` to decode numbers as `json.Number` (a string-backed type) instead of `float64`:

```
dec := json.NewDecoder(bytes.NewReader(jsonBytes))
dec.UseNumber()
var data any
dec.Decode(&data)
m := data.(map[string]any)
count, _ := m["count"].(json.Number).Int64()
```

Predeclared Identifiers Can Be Shadowed

All predeclared identifiers can be shadowed by local declarations. Go does not treat them as reserved keywords — they are ordinary identifiers that happen to be in scope by default.

Predeclared types (20):

`string`, `bool`, `int`, `int8`, `int16`, `int32`, `int64`, `uint`, `uint8`, `uint16`, `uint32`, `uint64`, `uintptr`, `byte`, `rune`, `float32`, `float64`, `complex64`, `complex128`, `error`, `any`

Predeclared constants (3):

`true`, `false`, `nil`

Predeclared functions (16):

`append`, `cap`, `close`, `complex`, `copy`, `delete`, `imag`, `len`, `make`, `new`, `panic`, `print`, `println`, `real`, `recover`

Any of these can be shadowed:

```
func check() bool {
    true := false  // legal but flagged by go vet
    return true    // returns false
}

func check() bool {
    int := 5       // legal, not flagged by go vet
    return true
}
```

`go vet` flags shadowing of `true`, `false`, and `nil`. Shadowing the rest is not flagged.